

Quantum Algorithms

Fourier Transform, Phase Estimation, Period Finding

The first six lectures of this course built a machine. They established what a qubit is and what it is made of, the linear algebra that describes it, and the fact that a small set of gates suffices to realise any unitary at all. *But what is the machine good for?*

These three lectures answer it. They are organised around a single mathematical object, the quantum Fourier transform, and around the observation that a quantum computer is very bad at telling you a list of numbers and very good at telling you a *phase*. Lecture 7 builds the transform and shows, on deliberately artificial problems, where a speed-up can come from at all. Lecture 8 turns the transform into a measuring instrument for phases. Lecture 9 points that instrument at a question in elementary number theory and, almost as a side effect, breaks the cryptosystem that secures most of the internet.

Cheat Sheet

Conventions used throughout these three lectures. Getting these wrong is the commonest source of error, so they are collected first.

Symbol	Meaning
$n, N = 2^n$	number of qubits in a Fourier register, and its dimension (Lectures 7–8)
$N, L = \lceil \log_2 N \rceil$	<i>in Lecture 9 only</i> : the integer to be factored, and its bit length
t	size of the counting register in phase estimation; the Fourier modulus is 2^t
ω_N	$e^{2\pi i/N}$, a primitive N th root of unity
bit order	$j = j_1 j_2 \cdots j_n$ with j_1 <i>most</i> significant, $j = \sum_l j_l 2^{n-l}$; qubit 1 at the top of a circuit
binary fraction	$0.j_1 j_2 \cdots j_n = \sum_{m \geq 0} j_{l+m} 2^{-(m+1)}$; e.g. $0.101 = 5/8$
Qiskit	uses the <i>opposite</i> order (j_0 least significant, at the top). Choose one and check on $n = 2$
$x \cdot y$	$\sum_j x_j y_j \bmod 2$, the binary inner product
$U_f, U_f^\pm$	Boolean oracle $ x\rangle y\rangle \mapsto x\rangle y \oplus f(x)\rangle$; phase oracle $ x\rangle \mapsto (-1)^{f(x)} x\rangle$

Lecture 7: complexity, and the Fourier transforms.

Scaling	Name	Example
$\mathcal{O}(1)$	constant	one query of Deutsch–Jozsa or Bernstein–Vazirani
$\mathcal{O}(\log n)$	logarithmic	binary search
$\mathcal{O}(n)$	linear	addition; queries used by Simon’s algorithm
$\mathcal{O}(n^2)$	quadratic	schoolbook multiplication; the quantum Fourier transform
$\mathcal{O}(n^3)$	cubic	Gaussian elimination; modular exponentiation; Shor overall
$\mathcal{O}(2^n)$	exponential	classical Deutsch–Jozsa with certainty; a general lookup table

Result	Statement
Hadamard layer	$H^{\otimes n} s\rangle = 2^{-n/2} \sum_x (-1)^{s \cdot x} x\rangle$, the Fourier transform of $(\mathbb{Z}_2)^n$
DFT	$\text{DFT} j\rangle = N^{-1/2} \sum_k \omega_N^{jk} k\rangle$; unitary; inverse has ω_N^{-jk}
roots of unity	$\sum_{k=0}^{N-1} \omega_N^{ak} = N$ if $a \equiv 0 \pmod{N}$, else 0 (<i>the one lemma</i>)
periodic vector	support $\{x_0, x_0 + r, \dots\}$ with $r \mid N \mapsto$ support $\{0, N/r, 2N/r, \dots\}$; x_0 becomes a phase
leakage	if $r \nmid N$ the peaks sit near, not on, the multiples of N/r , and weight spreads
FFT	classical DFT costs $\mathcal{O}(N \log N) = \mathcal{O}(n 2^n)$
QFT	same map as the DFT; $n = 1$ is exactly H
product form	$\text{QFT} j\rangle = \bigotimes_{l=1}^n (0\rangle + e^{2\pi i 0.j_{n-l+1} \cdots j_n} 1\rangle) / \sqrt{2}$ (a <i>product</i> state)
QFT circuit	H and controlled $R_k = \text{diag}(1, e^{2\pi i/2^k})$, then swaps; $n(n+1)/2 = \mathcal{O}(n^2)$ gates
approximate QFT	drop $k > k_{\max}$: error $< 2\pi n/2^{k_{\max}}$; $k_{\max} = \mathcal{O}(\log n)$ gives $\mathcal{O}(n \log n)$ gates
the rule	efficient QFT $\neq$ efficient DFT: you need <i>cheap read-in and cheap read-out</i>
practicality	small rotations are below hardware resolution; circuit needs all-to-all connectivity

Lecture 8: phase estimation. Given $U|u\rangle = e^{2\pi i \varphi}|u\rangle$, find φ .

Quantity	Statement
the circuit	$H^{\otimes t}$, then $C\text{-}U^{2^j}$ for $j = 0, \dots, t-1$, then $\text{QFT}^\dagger$, then measure
phase kickback	$C\text{-}U^{2^k}$ on $ +\rangle u\rangle$ gives $(0\rangle + e^{2\pi i 2^k \varphi} 1\rangle) / \sqrt{2} \otimes u\rangle$
exact case	if $\varphi = 0.\varphi_1 \cdots \varphi_t$, the register holds $\text{QFT} 2^t \varphi\rangle$; $\text{QFT}^\dagger$ gives the bits with certainty
general case	$p(\ell) = 2^{-2t} \sin^2(\pi 2^t \delta_\ell) / \sin^2(\pi \delta_\ell)$, $\delta_\ell = \varphi - \ell/2^t$
best-estimate bound	$p(\text{nearest } t\text{-bit value}) \geq 4/\pi^2 \approx 0.405$, independent of φ and t
register size	n correct bits with probability $\geq 1 - \epsilon$ needs $t = n + \lceil \log_2(2 + 1/2\epsilon) \rceil$
superposed input	$\sum_u c_u u\rangle$ returns φ_u with probability $ c_u ^2$, leaving $ u\rangle$ behind
cost	$\mathcal{O}(t^2)$ gates for $\text{QFT}^\dagger$, but $2^t - 1$ applications of U <i>in sequence</i>
Hadamard test	one ancilla, $p(0) = \cos^2(\pi \varphi)$; precision ϵ costs $\mathcal{O}(1/\epsilon^2)$ runs
iterative PE	one ancilla reused t times with classical feedback; same U count, far less coherence
Grover operator	$G = U_s U_f^\pm$ rotates by 2θ with $\sin \theta = \sqrt{M/N}$; eigenvalues $e^{\pm 2i\theta}$
search / counting	search: $k \approx \frac{\pi}{4} \sqrt{N/M}$ iterations (<i>quadratic</i>); counting: phase estimation on G
amplitude estimation	$a = \ \Pi A 0\rangle\ ^2$ to precision ϵ with $\mathcal{O}(1/\epsilon)$ uses of A , not $\mathcal{O}(1/\epsilon^2)$

Lecture 9: number theory, order finding, and Shor.

Object	Statement
order	$r = \text{ord}_N(x)$ is the least $r > 0$ with $x^r \equiv 1 \pmod N$; requires $\gcd(x, N) = 1$; $r < N$
Euclid	$\gcd(a, b) = \gcd(b, a \bmod b)$; cost $\mathcal{O}(L^2)$; also gives modular inverses
RSA	$N = pq$, $ed \equiv 1 \pmod{\phi(N)}$, $C = M^e$, $M = C^d$; factoring N yields d
GNFS	best classical factoring: $\exp(c L^{1/3} (\log L)^{2/3})$
Theorem 1	$y^2 \equiv 1 \pmod N$, $1 < y < N - 1 \Rightarrow \gcd(y \pm 1, N)$ is a non-trivial factor
Theorem 2	random $x \in \mathbb{Z}_N^*$, N odd with m distinct primes: $\Pr[r \text{ even and } x^{r/2} \not\equiv -1] \geq 1 - 2^{1-m}$
the six steps	prime? even? perfect power? random x and $\gcd$; order find ; $\gcd(x^{r/2} \pm 1, N)$
U_x	$ y\rangle \mapsto xy \bmod N\rangle$ for $y < N$, identity above; unitary because $\gcd(x, N) = 1$
eigenstates	$ u_s\rangle = r^{-1/2} \sum_k e^{-2\pi i s k / r} x^k \bmod N\rangle$, $U_x u_s\rangle = e^{2\pi i s / r} u_s\rangle$
the trick	$r^{-1/2} \sum_{s=0}^{r-1} u_s\rangle = 1\rangle$ (one X gate); gives a uniformly random s
controlled powers	$U_x^{2^j}$ is multiplication by $x^{2^j} \bmod N$, squared <i>classically</i> in advance
whole block	$ z\rangle y\rangle \mapsto z\rangle x^z y \bmod N\rangle$: modular exponentiation, $\mathcal{O}(L^3)$ gates
register sizes	counting $t = 2L + \mathcal{O}(1)$; work L ; plus $\mathcal{O}(L)$ ancillas for the arithmetic
continued fractions	$ s/r - \varphi \leq 1/2r^2 \Rightarrow s/r$ is a convergent of φ ; hence $t \approx 2L$
failure modes	bad estimate; $\gcd(s, r) > 1$; r odd; $x^{r/2} \equiv -1$. All caught by testing $x^{r'} \equiv 1$
total cost	$T_{\text{Shor}}(L) = \mathcal{O}(L^3)$, or $\mathcal{O}(L^2 \log L \log \log L)$ with fast multiplication
hidden subgroup	DJ, BV, Simon, order finding and discrete log are one problem; abelian solved, rest open

Small numbers worth remembering.

$4/\pi^2 = 0.4053$	worst-case success probability of one round of phase estimation
$N = 15$	every order is 1, 2 or 4: all phases exact, continued fractions trivial. A degenerate example
$N = 21, x = 2$	$r = 6, L = 5, t = 11$; $\varphi \approx 341/2048$; convergents $0/1, 1/6, 170/1021$; $y = 8$; $21 = 3 \times 7$
RSA-2048	$\sim 10^6$ physical qubits and hours of runtime, <i>with</i> error correction
today's hardware	$\sim 10^3$ physical qubits, $\sim 10^3$ – 10^5 gates before decoherence

Part I

Lecture 7: The Fourier Transform

Everything so far has been about capability. We know that a qubit exists, that its state is a vector, that gates are unitaries, and that a handful of gates can build any unitary at all. A machine that can do anything is not yet a machine that is *useful*, and the gap between those two statements is the subject of the next three lectures.

The plan is narrow on purpose. We will not survey quantum algorithms; we will follow one object, the Fourier transform, from a definition to Shor's factoring algorithm. Along the way almost every idea that makes a quantum algorithm work will appear at least once: superposition as a way of querying a function everywhere at once, interference as the only way of getting an answer back out, and the hard discipline of arranging for the answer you want to be the only thing that survives the interference.

1 Block 1: What “Faster” Means, and Where a Speed-up Comes From

Learning goals:

- State the definition of $\mathcal{O}$ notation, and say what “efficient” means for an algorithm.
- Distinguish query complexity from gate complexity, and explain why a promise problem is a legitimate computational question.
- Compute the action of $H^{\otimes n}$ on an arbitrary computational basis state.
- Analyse the Deutsch–Jozsa and Bernstein–Vazirani algorithms and state the speed-up each achieves.
- Explain Simon's algorithm, including its classical post-processing, and say why it is the template for everything in Lecture 9.

A short raison d'être

There is a story about quantum computing which says that a register of n qubits holds 2^n numbers at once, so a quantum computer evaluates a function on all 2^n inputs in parallel and is therefore exponentially fast. Every clause of that sentence is true except the last one, and the last one is false for a reason that this block is designed to make unforgettable: a measurement returns n bits, not 2^n amplitudes. Whatever the machine computed in superposition, you are allowed to keep one string of n bits of it.

An algorithm is therefore never a way of computing many answers. It is a way of arranging for the 2^n amplitudes to *interfere*, so that the single string you are permitted to read is a string you actually wanted. The three algorithms in this block do nothing else, and they are arranged in increasing order of how much that trick buys you: a certainty where classically you had a guess, then a factor of n , then a genuine exponential.

1.1 Counting Resources

Before we can say that one algorithm is faster than another we have to say what we are counting. Take the addition you were taught at school, applied to two n -digit numbers. Column by column you perform n single-digit additions, plus a carry now and then, so at most $2n$ elementary steps.

If you write the numbers in binary instead of decimal the count changes, because a number needs a different quantity of bits than digits, but it changes by a constant factor. If you count logic gates instead of digit additions, that too is a constant factor. If you count transistors, again a constant factor.

What survives every one of these changes of unit is the *scaling*: the cost of addition grows linearly with the size of the input, whatever the unit. That is the quantity worth naming.

Definition 1.1 (asymptotic notation). Let $f, g : \mathbb{N} \rightarrow \mathbb{R}$. We write

$$f(n) = \mathcal{O}(g(n)) \quad \text{if} \quad \exists C < \infty, n_0 < \infty \quad \text{such that} \quad |f(n)| \leq C |g(n)| \quad \forall n > n_0. \quad (1)$$

Note that $\mathcal{O}$ is an upper bound and need not be tight: $1 = \mathcal{O}(n)$ is a true and useless statement. Schoolbook multiplication of n -digit numbers costs $\mathcal{O}(n^2)$, and that bound is tight for the schoolbook method; there exist algorithms achieving $\mathcal{O}(n^{1+\epsilon})$ for any $\epsilon > 0$, and it is those, not the schoolbook method, that are asymptotically better. Yet your computer's arithmetic library does not use them for numbers of ordinary size, because the constant C they hide is enormous.

The ladder, and the word “efficient”.

Scaling	Name	Example
$\mathcal{O}(1)$	constant	compiling an AND gate into quantum gates
$\mathcal{O}(\log n)$	logarithmic	binary search in a sorted list
$\mathcal{O}(n)$	linear	addition of two n -bit numbers
$\mathcal{O}(n^2)$	quadratic	schoolbook multiplication; the quantum Fourier transform
$\mathcal{O}(n^3)$	cubic	Gaussian elimination; modular exponentiation
$\mathcal{O}(2^n)$	exponential	evaluating a general Boolean function by its lookup table

An algorithm is called *efficient* when its cost is $\mathcal{O}(n^p)$ for some finite p , and *inefficient* when it is exponential. Note that n is whatever measures the size of the input, and that it is not the same quantity in every row above: the number of entries in the list for binary search, the number of bits for the arithmetic entries, the dimension of the matrix for Gaussian elimination, the number of input bits for the lookup table. Two costs can only be compared once one has agreed what n counts, and Lecture 9 will turn on exactly this point.

Three different costs will matter to us and they are not interchangeable. *Gate complexity* (or *time complexity*, or the more limited *depth* or *2-qubit gate depth*) counts elementary gates; it is the closest analogue of running time (notwithstanding mid-circuit measurements). *Space complexity* counts qubits. *Query complexity* counts how many times the algorithm consults a given subroutine, treating everything else as free. Query complexity is the crudest of the three, but it is the one in which the cleanest separations between classical and quantum computation are known, and all three algorithms of this block are stated in it.

One more piece of vocabulary, used without proof. The class **P** contains the decision problems solvable in polynomial time by a classical computer; **NP** those whose *solutions* can be checked in polynomial time; **BQP** those solvable in polynomial time by a quantum computer with probability of error bounded below $1/2$. Every problem in these notes lies in **BQP**.

Two further statements are worth making carefully, because the opposite of each is often asserted.

First, factoring is *not* known to be **NP**-complete, and is not believed to be. Its decision version (“does N have a factor below k ?”) lies in $\mathbf{NP} \cap \mathbf{coNP}$, since a factorisation certifies a yes and, by Pratt’s theorem, also certifies a no. No problem in that intersection can be **NP**-

complete unless $\text{NP} = \text{coNP}$, which is widely disbelieved.¹ Factoring is believed to be one of the problems that are neither in P nor NP -complete, and this matters for the present course: Shor’s algorithm attacks a problem with unusually rich structure, and gives no reason to expect a quantum attack on problems that have none.

Second, no quantum algorithm is known that solves an NP -complete problem in polynomial time. The speed-ups known for such problems are polynomial, and usually no better than the quadratic one we shall meet in Lecture 8, Section 6.2. There is also evidence that no better is possible: any quantum algorithm that searches 2^n possibilities through a black box, using no structure beyond the ability to test a candidate, needs about $2^{n/2}$ queries.² Quantum computers are not believed to solve NP -complete problems efficiently.

1.2 The Query Model

The algorithms of this block are all of the following shape. We are handed a function $f : \{0,1\}^n \rightarrow \{0,1\}$ as a black box, together with a *promise* that f belongs to some restricted family, and we are asked one question about f globally. We may evaluate f as often as we like; we pay one unit per evaluation and nothing for anything else.

Lecture 6 established how such a black box is realised on a quantum computer, and we only recall the result. The *Boolean oracle* is the unitary

$$U_f |x\rangle \otimes |y\rangle = |x\rangle \otimes |y \oplus f(x)\rangle, \quad (2)$$

which is reversible because $\oplus$ is its own inverse, and the *phase oracle* is

$$U_f^\pm |x\rangle = (-1)^{f(x)} |x\rangle. \quad (3)$$

The second is built from the first by setting the target register to $|-\rangle$: since $X|-\rangle = -|-\rangle$, equation (2) gives $U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$, and the target factors out untouched. This is *phase kickback*, and Lecture 5 derived it in the special case of CNOT acting on $|+-\rangle$. Lecture 6 also showed that any f computable by a classical circuit of g gates can be realised as a U_f of $\mathcal{O}(g)$ gates and no leftover garbage, by computing, copying the answer with CNOTs, and uncomputing.

Two caveats. A promise is not a cheat, since real problems come with promises all the time, and an algorithm may assume one provided it does not quietly use it to make the problem trivial. And the query count hides the cost of building the black box, which is why Deutsch–Jozsa is elegant rather than useful, and why Lecture 9, where the oracle is modular exponentiation, will count its gates.

1.3 Hadamards on a Bit String

Every algorithm in this lecture begins and ends with a layer of Hadamard gates, so we need to know exactly what such a layer does. Lecture 4, Block 4 derived the special case $H^{\otimes n} |0\rangle^{\otimes n} = 2^{-n/2} \sum_x |x\rangle$. Here is the general statement.

¹S. Aaronson, “ $\text{P} \stackrel{?}{=} \text{NP}$ ”, in *Open Problems in Mathematics*, Springer (2016), §2.2.4, which states the argument and its consequence explicitly: factoring cannot be NP -complete unless $\text{NP} = \text{coNP}$.

²C. H. Bennett, E. Bernstein, G. Brassard and U. Vazirani, “Strengths and weaknesses of quantum computing”, *SIAM Journal on Computing* **26**, 1510 (1997); arXiv:quant-ph/9701001. The bound holds relative to an oracle, so it does not settle whether $\text{NP} \subseteq \text{BQP}$; what it does rule out is any purely brute-force quantum attack, which is what most informal claims amount to. For a survey of the wider question see S. Aaronson, “NP-complete problems and physical reality”, *SIGACT News* **36**(1), 30 (2005).

Proposition 1.2. For any bit string $s \in \{0,1\}^n$,

$$H^{\otimes n} |s\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} (-1)^{s \cdot x} |x\rangle, \quad \text{where} \quad s \cdot x = \sum_{j=1}^n s_j x_j \bmod 2. \quad (4)$$

Proof. On a single qubit, $H |s_j\rangle = \frac{1}{\sqrt{2}} \sum_{x_j \in \{0,1\}} (-1)^{s_j x_j} |x_j\rangle$: for $s_j = 0$ both signs are +, and for $s_j = 1$ the $|1\rangle$ component acquires a minus sign, which is precisely what $(-1)^{s_j x_j}$ encodes. Taking the tensor product over the n qubits and using the mixed-product property of Lecture 4, Block 4,

$$H^{\otimes n} |s\rangle = \bigotimes_{j=1}^n \frac{1}{\sqrt{2}} \sum_{x_j} (-1)^{s_j x_j} |x_j\rangle = \frac{1}{2^{n/2}} \sum_{x_1 \cdots x_n} (-1)^{\sum_j s_j x_j} |x_1 \cdots x_n\rangle,$$

which is (4). The reduction of the exponent modulo 2 is automatic: -1 raised to an integer power only sees that integer's parity. $\square$

Read equation (4) twice. The first reading is the operational one: a layer of Hadamards turns a bit string into a uniform superposition whose *signs* record the string. The second reading is the one that organises this whole lecture. The signs $(-1)^{s \cdot x}$ in (4) have the property that

$$(-1)^{s \cdot (x \oplus y)} = (-1)^{s \cdot x} (-1)^{s \cdot y},$$

that is, they convert the natural operation on bit strings, which is $\oplus$, into ordinary multiplication. In Block 2 we shall meet the phases $e^{2\pi i j k / N}$, which do the same for addition modulo N . A transform built out of such phases is what is meant by a Fourier transform.³ So:

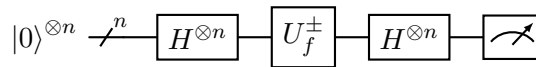
$H^{\otimes n}$ is a Fourier transform: the Fourier transform over the group $(\mathbb{Z}_2)^n$. Every algorithm in Block 1 is therefore already a Fourier algorithm, and the work of Blocks 2 and 3 is to build the transform for the group $\mathbb{Z}_N$ instead, which is the one that knows about periods.

1.4 Deutsch–Jozsa

Definition 1.3 (the Deutsch–Jozsa problem). We are given $f : \{0,1\}^n \rightarrow \{0,1\}$ with the promise that f is either *constant* ($f(x)$ is the same for every x) or *balanced* ($f(x) = 0$ for exactly half the inputs and 1 for the other half). Decide which.

Classically, and with certainty, this is expensive. Each query returns one bit. Having seen 2^{n-1} inputs all giving the same answer we still cannot conclude, because a balanced function is free to return 0 on exactly those inputs and 1 on all the others; only the next query settles it. So $2^{n-1} + 1$ queries are both necessary and sufficient, and the classical cost therefore grows as 2^n . (If we are content with high probability rather than certainty, a handful of random queries suffice; this is the first hint that the separation is fragile.)

The quantum algorithm uses one query. Prepare $|0\rangle^{\otimes n}$, apply $H^{\otimes n}$, apply the phase oracle, apply $H^{\otimes n}$ again, and measure.



³In the language of group theory, $\{0,1\}^n$ under $\oplus$ is the group $(\mathbb{Z}_2)^n$, and the functions $\chi_s(x) = (-1)^{s \cdot x}$ are its *characters*; $e^{2\pi i j k / N}$ are likewise the characters of $\mathbb{Z}_N$. Nothing below depends on knowing this vocabulary.

After the first layer and the oracle the state is, by (4) with $s = 0$ and then (3),

$$|\psi\rangle = \frac{1}{2^{n/2}} \sum_x (-1)^{f(x)} |x\rangle. \quad (5)$$

We only need the probability of the outcome $0 \cdots 0$. Applying the final Hadamard layer and using (4) once more, the amplitude of $|0\rangle^{\otimes n}$ is

$$\langle 0 \cdots 0 | H^{\otimes n} \psi \rangle = \frac{1}{2^n} \sum_x (-1)^{f(x)}, \quad \text{so} \quad p(0 \cdots 0) = \left| \frac{1}{2^n} \sum_x (-1)^{f(x)} \right|^2. \quad (6)$$

Now the promise does all the work. If f is constant, every term in the sum is the same sign, the sum is $\pm 2^n$, and $p(0 \cdots 0) = 1$. If f is balanced, exactly half the terms are $+1$ and half are -1 , the sum is *zero*, and $p(0 \cdots 0) = 0$.

So: measure. All zeros means constant; anything else means balanced. One query, with certainty, against $2^{n-1} + 1$ classically.

It is worth being precise about what happened, because it is the mechanism of every later algorithm. The oracle did not compute 2^n values for us to read. It wrote 2^n signs into the amplitudes of a single state, and the final Hadamard layer added those signs up. The quantity $2^{-n} \sum_x (-1)^{f(x)}$ is a *global* property of f , from which no single value of f can be recovered, and it is exactly the kind of property that destructive interference can compute in one shot.

1.5 Bernstein–Vazirani

Definition 1.4 (the Bernstein–Vazirani problem). We are given $f(x) = s \cdot x \bmod 2$ for some unknown $s \in \{0, 1\}^n$. Find s .

Classically this is not catastrophic, merely linear: query f on $e_j = 0 \cdots 010 \cdots 0$, the string with a single 1 in position j , and read off $f(e_j) = s_j$. After n queries we know s . And n queries are necessary, since each returns one bit and s carries n .

The quantum algorithm is the circuit of Section 1.4, unchanged, and the analysis is two lines. By (5) the state after the oracle is

$$\frac{1}{2^{n/2}} \sum_x (-1)^{s \cdot x} |x\rangle = H^{\otimes n} |s\rangle,$$

where the second equality is nothing but Proposition 1.2 read from right to left. A final Hadamard layer therefore returns $H^{\otimes n} H^{\otimes n} |s\rangle = |s\rangle$, and a measurement yields s with certainty. One query instead of n .

Remark 1.5. The separation here is $n \rightarrow 1$, not exponential, and Bernstein–Vazirani is often dismissed for that reason. The dismissal misses the point. Deutsch–Jozsa extracted one bit of global information; Bernstein–Vazirani extracts all n bits of a hidden string in a single query, because the string was hidden in the *Fourier domain* and the final Hadamard layer is precisely the transform that brings it back. That is a template, and the next algorithm runs it to its conclusion.

1.6 Simon’s Problem

Definition 1.6 (Simon’s problem). We are given $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with the promise that there exists $c \in \{0, 1\}^n$ such that

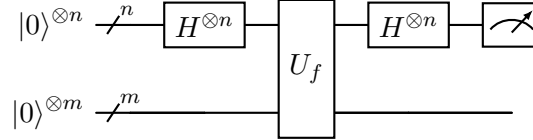
$$f(x) = f(y) \iff y = x \text{ or } y = x \oplus c. \quad (7)$$

Find c . (If $c = 0 \cdots 0$ the function is one-to-one; otherwise it is two-to-one.)

The string c is a *period*: $f(x \oplus c) = f(x)$ for every x , and c is the only nonzero string with that property. Simon's problem is the question "what is the period of f ?" asked in the group $(\mathbb{Z}_2)^n$, where "adding the period" means $\oplus$.

Classically one is reduced to hunting for a collision. Having queried k distinct inputs, one has $\binom{k}{2}$ pairs, each of which collides only if the two inputs happen to differ by c ; a birthday-paradox count shows that about $2^{n/2}$ queries are needed before a collision is even likely. This is a proven lower bound, not merely the best known strategy: classically the problem requires at least of order $2^{n/2}$ queries.

The quantum algorithm uses two registers: n qubits for the input and m for the output.



Step 1. After the Hadamard layer and the oracle,

$$\frac{1}{2^{n/2}} \sum_x |x\rangle |f(x)\rangle.$$

Step 2. We do not measure the second register, but it is clarifying to imagine that we do. Suppose the outcome is some value $f(x_0)$. By the promise exactly two inputs produce it, so the first register collapses to

$$\frac{1}{\sqrt{2}}(|x_0\rangle + |x_0 \oplus c\rangle). \quad (8)$$

This state contains c , but not in a form we can read: measuring it returns x_0 or $x_0 \oplus c$ with equal probability, and x_0 is random and unknown, so the outcome is a uniformly random string carrying no information at all. This is the characteristic obstruction of the whole subject. The answer is present in the state, hidden behind an unknown offset.

Step 3. Apply $H^{\otimes n}$ to (8). By Proposition 1.2,

$$\begin{aligned} \frac{1}{\sqrt{2}} \left(H^{\otimes n} |x_0\rangle + H^{\otimes n} |x_0 \oplus c\rangle \right) &= \frac{1}{2^{(n+1)/2}} \sum_y \left[(-1)^{x_0 \cdot y} + (-1)^{(x_0 \oplus c) \cdot y} \right] |y\rangle \\ &= \frac{1}{2^{(n+1)/2}} \sum_y (-1)^{x_0 \cdot y} \left[1 + (-1)^{c \cdot y} \right] |y\rangle, \end{aligned}$$

where we used $(x_0 \oplus c) \cdot y = x_0 \cdot y + c \cdot y \pmod{2}$. The unknown offset x_0 has been demoted to an overall sign $(-1)^{x_0 \cdot y}$ on each term, which is invisible to a measurement. And the bracket is 2 when $c \cdot y = 0$ and 0 when $c \cdot y = 1$.

Step 4. Measure the first register. The outcome is a string y , drawn uniformly from the 2^{n-1} strings satisfying

$$c \cdot y = 0 \pmod{2}. \quad (9)$$

That is one linear equation for the n unknown bits of c , over the field $\mathbb{Z}_2$.

Step 5. Repeat. Each run costs one query and yields one random equation. After about n runs we hold n equations; with probability bounded below by a constant they include $n - 1$ linearly independent ones, which determine c up to the two solutions 0 and c itself. Solving the system is Gaussian elimination modulo 2, a classical $\mathcal{O}(n^3)$ computation. Finally check the answer: evaluate $f(0)$ and $f(c)$, and accept c if they agree.

The total cost is $\mathcal{O}(n)$ queries and $\mathcal{O}(n^3)$ classical post-processing, against at least of order $2^{n/2}$ queries classically. This is an *exponential* separation, and it was the first one known.

The shape of the argument, which is the shape of Lecture 9.

1. Prepare a uniform superposition and query the function once, producing the state (8), supported on the pair $\{x_0, x_0 \oplus c\}$: the hidden period c is present, but displaced by an unknown offset x_0 .
2. That unknown offset makes a direct measurement useless.
3. A Fourier transform turns the offset into a phase, which no measurement can see, and leaves the period visible in *which* outcomes have nonzero probability.
4. Measure, obtaining one random constraint on the hidden object; repeat a few times and finish the job with classical algebra.

Shor's algorithm is these same four steps with $(\mathbb{Z}_2)^n$ replaced by $\mathbb{Z}_N$, " $\oplus c$ " replaced by " $+r$ ", and Gaussian elimination replaced by continued fractions. The only genuinely new ingredient is the Fourier transform of step 3, because over $\mathbb{Z}_N$ it is no longer a layer of Hadamards. Building it is the business of the rest of this lecture.

And the moral of the block. Quantum parallelism is not the resource. In all three algorithms the oracle was queried once, on a superposition, and the information so obtained was inaccessible until a Fourier transform made one particular global feature (a parity, a hidden string, a hidden period) interfere constructively while everything else cancelled. Speed-ups come from hidden global structure, not from doing many things at once.

2 Block 2: The Discrete Fourier Transform

Learning goals:

- State the definition of the discrete Fourier transform and prove that it is unitary.
- Prove the geometric-sum lemma for roots of unity and recognise it as the computational engine of every algorithm in these three lectures.
- Describe what the DFT does to a vector supported on an arithmetic progression, in both the commensurate and incommensurate cases.
- State the cost of the fast Fourier transform and explain why it is the correct classical baseline.

A short raison d'être

The Fourier transform is the instrument by which periodicity becomes visible. A function that repeats is, in the time domain, a complicated object about which one must ask "how long until it repeats?"; in the frequency domain it is a spike, and the question has become "where is the spike?". Since a quantum computer is permitted to hand us the location of exactly one spike, and since Block 1 reduced a hidden-period problem to a Fourier transform, this is the transform we need.

This block is entirely classical. It is here because the quantum circuit of Block 3 is meaningless without it, and because the two facts that will carry Lecture 9, namely that sums of roots of unity vanish and that a periodic vector transforms to a comb, are facts about the ordinary DFT that have nothing to do with quantum mechanics.

2.1 From the Integral to the Matrix

The Fourier transform you already know takes a function and returns its spectrum,

$$\hat{f}(k) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i k t} dt,$$

and is useful whenever a signal has periodic structure that must be found or manipulated. The discrete version replaces the function by a finite list of numbers. Let N be a positive integer and write

$$\omega_N = e^{2\pi i/N}, \quad (10)$$

a primitive N th root of unity, so that $\omega_N^N = 1$ and the powers $\omega_N^0, \dots, \omega_N^{N-1}$ are N distinct points spaced evenly around the unit circle.

Definition 2.1 (discrete Fourier transform). The DFT is the linear map on $\mathbb{C}^N$ defined on the standard basis by

$$\text{DFT} : |j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega_N^{jk} |k\rangle, \quad j = 0, 1, \dots, N-1, \quad (11)$$

and hence on a general vector $|f\rangle = \sum_j f_j |j\rangle$ by

$$\text{DFT} |f\rangle = \sum_{k=0}^{N-1} \hat{f}_k |k\rangle, \quad \hat{f}_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \omega_N^{jk} f_j. \quad (12)$$

We use Dirac notation for these vectors out of habit and convenience; nothing here requires $|f\rangle$ to be a quantum state. The symmetric normalisation $1/\sqrt{N}$ is chosen so that the transform is unitary, which is the only reason a quantum computer can hope to implement it.

Proposition 2.2. DFT is unitary, and $\text{DFT}^\dagger$ is given by (11) with ω_N replaced by $\omega_N^{-1} = \overline{\omega_N}$.

Proof. Write F for the matrix with entries $F_{kj} = N^{-1/2} \omega_N^{jk}$. Then

$$(F^\dagger F)_{j'j} = \sum_k \overline{F_{kj'}} F_{kj} = \frac{1}{N} \sum_{k=0}^{N-1} \omega_N^{-j'k} \omega_N^{jk} = \frac{1}{N} \sum_{k=0}^{N-1} \omega_N^{(j-j')k}.$$

By Lemma 2.3 below this equals 1 when $j = j'$ and 0 otherwise, so $F^\dagger F = I$. Since F is square this makes it unitary, and $F^{-1} = F^\dagger$ has entries $N^{-1/2} \omega_N^{-jk}$. $\square$

2.2 Roots of Unity: the One Lemma

Lemma 2.3 (geometric sum of roots of unity). For any integer a ,

$$\sum_{k=0}^{N-1} \omega_N^{ak} = \begin{cases} N, & a \equiv 0 \pmod{N}, \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

Proof. If $a \equiv 0$ modulo N then $\omega_N^a = 1$ and every term of the sum is 1, giving N . Otherwise $\omega_N^a \neq 1$ and the sum is geometric:

$$\sum_{k=0}^{N-1} (\omega_N^a)^k = \frac{\omega_N^{aN} - 1}{\omega_N^a - 1} = \frac{1 - 1}{\omega_N^a - 1} = 0,$$

since $\omega_N^{aN} = (\omega_N^N)^a = 1$. $\square$

2.3 The Transform of a Periodic Vector

Here is the reason the Fourier transform is the right tool for period finding. Let r divide N , let $0 \leq x_0 < r$, and consider the vector supported on the arithmetic progression $x_0, x_0 + r, x_0 + 2r, \dots$:

$$|f\rangle = \sqrt{\frac{r}{N}} \sum_{m=0}^{N/r-1} |x_0 + mr\rangle. \quad (14)$$

This is exactly the state that emerges from a period-finding oracle: measure the output register and the input register is left on the set of inputs consistent with that output, which is a progression of step r starting at some unknown x_0 .

Proposition 2.4. *With $r \mid N$, the DFT of (14) is*

$$\text{DFT}|f\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \omega_N^{x_0 s N/r} \left| s \frac{N}{r} \right\rangle. \quad (15)$$

Proof. By (12),

$$\hat{f}_k = \frac{1}{\sqrt{N}} \sqrt{\frac{r}{N}} \sum_{m=0}^{N/r-1} \omega_N^{(x_0+mr)k} = \frac{\sqrt{r}}{N} \omega_N^{x_0 k} \sum_{m=0}^{N/r-1} (\omega_N^{rk})^m.$$

Now $\omega_N^{rk} = e^{2\pi i rk/N} = \omega_{N/r}^k$, so the sum over m is exactly the sum of Lemma 2.3 with modulus N/r . It equals N/r when k is a multiple of N/r and 0 otherwise. Writing $k = sN/r$ gives $\hat{f}_k = \sqrt{r} \omega_N^{x_0 k} / r = \omega_N^{x_0 s N/r} / \sqrt{r}$, and (15) follows. $\square$

Three features of (15) deserve to be read off explicitly, because all three are used in Lecture 9.

A comb becomes a comb. The input was supported on N/r points spaced r apart; the output is supported on r points spaced N/r apart. Narrow in position means wide in frequency and conversely, which is the uncertainty principle in its most elementary form.

The offset becomes a phase. The unknown x_0 appears in (15) only in the phases $\omega_N^{x_0 s N/r}$. Since a measurement sees $|\hat{f}_k|^2$, the offset is invisible. This is exactly what happened in Simon's algorithm at step 3, and it is why the transform is worth doing: the thing we do not know has been made irrelevant and the thing we want has been made visible.

The measurement returns a random multiple of N/r . The outcome is $k = sN/r$ with s uniform on $\{0, \dots, r-1\}$. From k we learn $s/r = k/N$, which determines r once we remove common factors from s and r . That final step, recovering r from a random multiple of $1/r$, will be performed by continued fractions in Lecture 9, Block 9.

Example 2.5 ($N = 8$, exact and inexact). Take $N = 8$ and $x_0 = 0$. With $r = 2$, the vector $\frac{1}{2}(|0\rangle + |2\rangle + |4\rangle + |6\rangle)$ transforms to $\frac{1}{\sqrt{2}}(|0\rangle + |4\rangle)$: two perfect spikes at multiples of $N/r = 4$, each of probability $1/2$.

With $r = 3$, however, $3 \nmid 8$: Proposition 2.4 does not apply, and the vector $\frac{1}{\sqrt{3}}(|0\rangle + |3\rangle + |6\rangle)$ transforms to a spectrum with probabilities

$$(0.375, 0.007, 0.042, 0.243, 0.042, 0.243, 0.042, 0.007).$$

The spikes have not disappeared. There are still three of them, near $k = 0$, $k \approx 8/3 = 2.67$ and $k \approx 16/3 = 5.33$, but since those positions are not integers the weight has smeared onto the neighbouring values. This is called *leakage*, and it is not an edge case to be avoided: in Lecture 9 the period r will be an unknown divisor of nothing in particular, so $r \nmid N$ will be the generic situation and every probability statement will have to survive it.

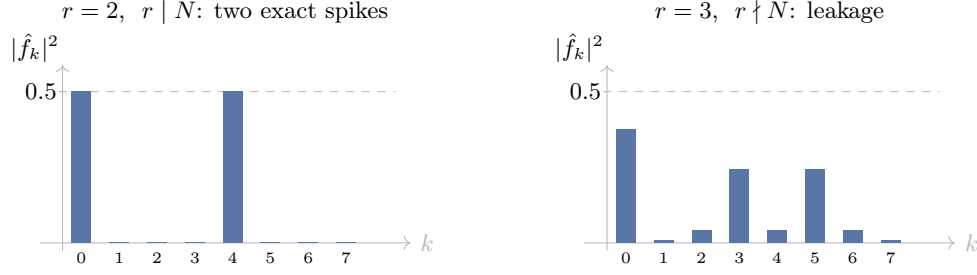


Figure 1: The DFT of a vector supported on $\{0, r, 2r, \dots\}$ in $\mathbb{C}^8$, as in Example 2.5. On the left $r = 2$ divides $N = 8$ and the spectrum is two clean spikes at multiples of $N/r = 4$. On the right $r = 3$ does not divide 8; the spectrum still concentrates near the multiples of $8/3$, but no longer exactly on integers, and the weight leaks onto neighbours. Lecture 9 lives in the right-hand picture.

2.4 The Fast Fourier Transform, and the Honest Baseline

How expensive is the DFT classically? Naively one multiplies an $N \times N$ matrix by a vector, at a cost of $\mathcal{O}(N^2)$ arithmetic operations. The fast Fourier transform does much better by splitting the sum (12) into its even and odd terms, recognising each half as a DFT of size $N/2$, and recursing; the recurrence $T(N) = 2T(N/2) + \mathcal{O}(N)$ gives

$$T(N) = \mathcal{O}(N \log_2 N) = \mathcal{O}(n 2^n), \quad N = 2^n, \quad (16)$$

where the second form is the first one rewritten: $N \log_2 N = 2^n \cdot n$.

This is the number to beat, and it is important to state it in both forms. Measured against the length N of the data, the FFT is essentially optimal, since one cannot do better than look at every entry. Measured against the number of *bits* n needed to index the data, it is exponential. The quantum Fourier transform of the next block will cost $\mathcal{O}(n^2)$, which is exponentially better in the second measure and, as we shall see, not directly comparable in the first.

2.5 Closing the Loop

Return to Proposition 1.2. The map $|s\rangle \mapsto 2^{-n/2} \sum_x (-1)^{s \cdot x} |x\rangle$ has exactly the form of Definition 2.1, with the phases ω_N^{jk} replaced by $(-1)^{s \cdot x}$. The replacement is not an analogy: $(-1)^{s \cdot x}$ is the general character of $(\mathbb{Z}_2)^n$ in precisely the way that ω_N^{jk} is the general character of $\mathbb{Z}_N$. So $H^{\otimes n}$ is the Fourier transform of the group $(\mathbb{Z}_2)^n$, and the DFT of Definition 2.1 is the Fourier transform of the cyclic group $\mathbb{Z}_N$.

Simon's algorithm succeeded because its hidden period lived in $(\mathbb{Z}_2)^n$, where the required transform happens to be n Hadamard gates and therefore costs nothing. The period hidden inside the factoring problem lives in $\mathbb{Z}_N$, where the required transform is Definition 2.1 and we will have to build it.

Looking ahead. Block 3 builds it, as a circuit of $\mathcal{O}(n^2)$ gates. The construction rests on a single algebraic surprise: although DFT $|j\rangle$ is a superposition of all N basis states, it is nevertheless a *product* state, with no entanglement between the qubits at all.

3 Block 3: The Quantum Fourier Transform

Learning goals:

- State the definition of the quantum Fourier transform on n qubits and its relation to the Hadamard gate.
- Use the binary-fraction notation fluently, and state the bit-ordering convention these notes adopt and how it differs from Qiskit's.
- Derive the product form of QFT $|j\rangle$.
- Construct the QFT circuit, count its gates, and explain the role of the final swaps.
- Explain, precisely, why an efficient quantum DFT does not give an efficient algorithm for computing discrete Fourier transforms of data.

A short raison d'être

This block contains a single calculation, equation (20), and everything else follows from it. The calculation is worth doing slowly, because its conclusion is genuinely surprising and because two lectures' worth of algorithms are corollaries of it.

The Fourier transform of a basis state is spread over all 2^n basis states, yet is not entangled at all. Each qubit ends up in an independent superposition of $|0\rangle$ and $|1\rangle$, differing only in a relative phase, and the phase on qubit l is a binary fraction built from the last l bits of the input. A product state is exactly what a circuit of one- and two-qubit gates can construct cheaply, and so the transform that costs $\mathcal{O}(n2^n)$ classically costs $\mathcal{O}(n^2)$ here.

3.1 Definition

Definition 3.1 (quantum Fourier transform). Let $N = 2^n$ and let the computational basis of n qubits be labelled by integers $|0\rangle, |1\rangle, \dots, |N-1\rangle$. The quantum Fourier transform is the unitary

$$\text{QFT } |j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k / N} |k\rangle. \quad (17)$$

It is the same map as Definition 2.1; the word “quantum” refers to the fact that we intend to realise it as a circuit, not to any change in the mathematics. Proposition 2.2 already tells us it is unitary, so a circuit is not forbidden.

For $n = 1$ we have $N = 2$, $\omega_2 = -1$, and

$$\text{QFT } |j\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^j |1\rangle),$$

which is the Hadamard gate. The single-qubit quantum Fourier transform is H . Everything in this block is the statement that the general case is not much worse.

3.2 Conventions: Bit Order and Binary Fractions

The derivation ahead is bookkeeping with indices, and the bookkeeping is only bearable with the right conventions. We adopt the following, and use them for the remaining two lectures.

An integer j with $0 \leq j < 2^n$ is written in binary as $j = j_1 j_2 \dots j_n$ with j_1 the *most* significant bit:

$$j = \sum_{l=1}^n j_l 2^{n-l} = j_1 2^{n-1} + j_2 2^{n-2} + \dots + j_n 2^0. \quad (18)$$

Correspondingly $|j\rangle = |j_1\rangle \otimes |j_2\rangle \otimes \dots \otimes |j_n\rangle$, with qubit 1 drawn at the top of a circuit diagram.

A *binary fraction* is a number less than one written in base two, exactly as a decimal fraction is a number less than one written in base ten. We write

$$0.j_l j_{l+1} \cdots j_n := \frac{j_l}{2} + \frac{j_{l+1}}{4} + \cdots + \frac{j_n}{2^{n-l+1}} = \sum_{m=0}^{n-l} \frac{j_{l+m}}{2^{m+1}}. \quad (19)$$

So, for instance, $0.101 = \frac{1}{2} + \frac{0}{4} + \frac{1}{8} = 0.625$ and $0.1 = \frac{1}{2}$.

The reason for putting the most significant bit first is (19): with this convention j_l carries weight 2^{-l} in a fraction, and dividing j by a power of two simply moves the binary point. Since the entire derivation below manipulates fractions rather than integers, this is the convention that keeps it readable.

Warning: this is the opposite of Qiskit's convention.

3.3 The Product Form

Theorem 3.2 (product form of the QFT). *With the conventions of Section 3.2 and $N = 2^n$,*

$$\text{QFT } |j_1 j_2 \cdots j_n\rangle = \bigotimes_{l=1}^n \frac{|0\rangle + e^{2\pi i 0.j_{n-l+1} \cdots j_n} |1\rangle}{\sqrt{2}}. \quad (20)$$

Written out, the first qubit of the output carries the phase $e^{2\pi i 0.j_n}$, the second $e^{2\pi i 0.j_{n-1} j_n}$, and the last $e^{2\pi i 0.j_1 j_2 \cdots j_n}$.

Proof. Start from (17) and expand the summation index k in binary, $k = k_1 k_2 \cdots k_n$, so that $\sum_{k=0}^{N-1} = \sum_{k_1=0}^1 \cdots \sum_{k_n=0}^1$ and $|k\rangle = |k_1\rangle \otimes \cdots \otimes |k_n\rangle$. By (18),

$$\frac{k}{N} = \frac{1}{2^n} \sum_{l=1}^n k_l 2^{n-l} = \sum_{l=1}^n k_l 2^{-l},$$

which is the binary fraction $0.k_1 k_2 \cdots k_n$: dividing by $N = 2^n$ has simply moved the binary point to the front. Therefore

$$\text{QFT } |j\rangle = \frac{1}{2^{n/2}} \sum_{k_1=0}^1 \cdots \sum_{k_n=0}^1 e^{2\pi i j \sum_{l=1}^n k_l 2^{-l}} |k_1\rangle \otimes \cdots \otimes |k_n\rangle = \frac{1}{2^{n/2}} \sum_{k_1=0}^1 \cdots \sum_{k_n=0}^1 \bigotimes_{l=1}^n e^{2\pi i j k_l 2^{-l}} |k_l\rangle,$$

where the exponential has been split across the tensor factors because the exponent is a sum with one term per factor. Each of the n sums now involves only its own tensor factor, so sums and tensor product may be interchanged:

$$\text{QFT } |j\rangle = \frac{1}{2^{n/2}} \bigotimes_{l=1}^n \left(\sum_{k_l=0}^1 e^{2\pi i j k_l 2^{-l}} |k_l\rangle \right) = \bigotimes_{l=1}^n \frac{|0\rangle + e^{2\pi i j 2^{-l}} |1\rangle}{\sqrt{2}}. \quad (21)$$

This is already a product state, which is the substantive point. It remains to simplify the phases.

Consider $j 2^{-l}$. Writing j in binary and moving the binary point l places,

$$\frac{j}{2^l} = j_1 \cdots j_{n-l} \cdot j_{n-l+1} \cdots j_n = \underbrace{(j_1 \cdots j_{n-l})}_{\text{an integer}} + 0.j_{n-l+1} \cdots j_n.$$

The integer part is irrelevant: $e^{2\pi i M} = 1$ for every integer M , so only the fractional part survives in $e^{2\pi i j 2^{-l}}$. Substituting into (21) gives (20). $\square$

Two consequences are worth stating separately.

No entanglement is created. The input $|j\rangle$ was a product state and so is the output. The transform redistributes information rather than correlating qubits: every bit of j influences the phase of several output qubits, and each output qubit's phase depends on several bits of j , but the joint state factorises completely. (This is a statement about basis states only. Applied to a superposition, QFT certainly produces entanglement, and a good thing too, since otherwise it could be simulated classically.)

The phases are nested. Qubit l of the output depends on the last l bits of the input. The first output qubit depends on j_n alone, and its phase is $e^{2\pi i 0.j_n}$, which is $+1$ if j is even and -1 if j is odd, so that qubit is obtained from $|j_n\rangle$ by a single Hadamard. The second depends on j_{n-1} and j_n , so it is a Hadamard plus one correction controlled by j_n . The pattern continues, and it is the circuit.

3.4 The Circuit

Define the single-qubit phase gate

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}, \quad (22)$$

so that $R_1 = Z$, $R_2 = S$ and $R_3 = T$. Its controlled version is not a primitive of any hardware, but Lecture 5 showed how to assemble a controlled rotation from CNOTs and single-qubit rotations. For R_k the assembly is Figure 2, and it is pleasingly economical: two CNOTs and three copies of R_{k+1} , the next gate down in the same family, one of them inverted.

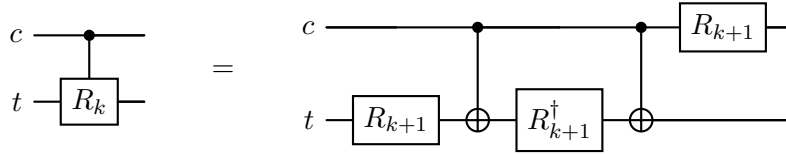


Figure 2: The controlled phase gate of the QFT circuit, written out. Since $R_k = \text{diag}(1, e^{i\theta})$ with $\theta = 2\pi/2^k$, and $R_{k+1} = \text{diag}(1, e^{i\theta/2})$, the right-hand circuit applies $e^{i\theta/2}$ to the target, then $e^{-i\theta/2}$ conditioned on the control by conjugating with CNOTs, and finally $e^{i\theta/2}$ to the control. The three contributions cancel except on $|11\rangle$, where they leave $e^{i\theta}$. Note that the controlled gate is *symmetric* between control and target, as the left-hand diagram's notation does not suggest.

The construction follows Theorem 3.2 term by term. To produce the output qubit carrying $e^{2\pi i 0.j_1 j_2 \dots j_n}$, start from $|j_1\rangle$ and apply H , which yields $(|0\rangle + e^{2\pi i 0.j_1} |1\rangle)/\sqrt{2}$; then note that

$$0.j_1 j_2 \dots j_n = 0.j_1 + 0.0j_2 + 0.00j_3 + \dots + 0.0 \dots 0j_n,$$

so the remaining bits contribute independent phase factors $e^{2\pi i j_2/4}$, $e^{2\pi i j_3/8}$, and so on. Each is applied by a controlled- R_k with the control on qubit k , which acts only when $j_k = 1$, exactly the condition for that term to be present. Repeating for each output qubit in turn gives the circuit

$$H_1, CR_2, \dots, CR_n \text{ on qubit 1; } H_2, CR_2, \dots, CR_{n-1} \text{ on qubit 2; } \dots; H_n \text{ on qubit } n, \quad (23)$$

followed by a layer of swaps. Figure 3 shows the case $n = 3$.

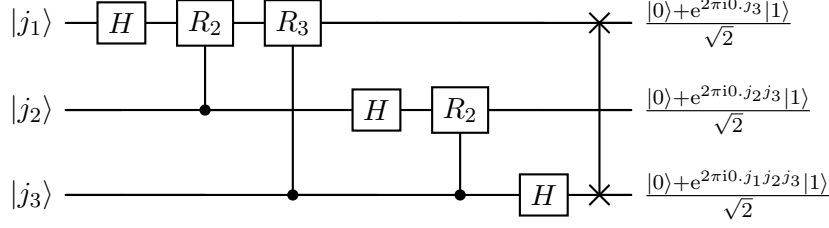


Figure 3: The quantum Fourier transform on three qubits. Immediately before the swap the wires carry the phases $0.j_1j_2j_3$, $0.j_2j_3$ and $0.j_3$ from top to bottom, which are the factors of (20) in *reverse* order. The swap restores the order, and the labels shown at the right are the output as (20) demands it.

Why the swaps. Compare the circuit's output with (20). The Hadamard-and-rotations network produces, on qubit 1, the factor whose phase is $0.j_1 \cdots j_n$, but Theorem 3.2 assigns that factor to the *last* tensor slot. The circuit computes the right state in the wrong order, reversed end to end, and $\lfloor n/2 \rfloor$ swap gates put it right. In an implementation one often omits the swaps and relabels the wires instead, which costs nothing; the only mistake is to forget that the issue exists.

Cost. Qubit 1 receives one Hadamard and $n - 1$ controlled rotations, qubit 2 receives $n - 2$, and so on, for a total of

$$n + [(n - 1) + (n - 2) + \cdots + 1] = n + \frac{n(n - 1)}{2} = \frac{n(n + 1)}{2} = \mathcal{O}(n^2) \quad (24)$$

gates, plus $\mathcal{O}(n)$ swaps. Against the $\mathcal{O}(n2^n)$ of the fast Fourier transform this is an exponential improvement, subject to caveats we must now state carefully.

3.5 The Inverse

By Proposition 2.2 the inverse transform is (17) with $e^{2\pi ijk/N}$ replaced by $e^{-2\pi ijk/N}$. As a circuit this means: run Figure 3 backwards, replacing each gate by its adjoint, so $R_k \rightarrow R_k^\dagger$ (that is, the phase $e^{2\pi i/2^k}$ becomes $e^{-2\pi i/2^k}$) and $H \rightarrow H$. The gate count is unchanged. We will use $\text{QFT}^\dagger$ far more often than QFT itself: from Lecture 8 onwards, every algorithm arranges for the answer to be *already* in the Fourier domain, and the job of the circuit is to bring it back.

3.6 The Approximate QFT

The controlled rotation CR_k applies a phase $2\pi/2^k$, which for large k is indistinguishable from doing nothing. Discard every rotation with $k > k_{\max}$. Each qubit then carries at most $k_{\max}$ rotations, so the circuit has $\mathcal{O}(nk_{\max})$ gates instead of $\mathcal{O}(n^2)$.

The error is easy to bound. Omitting a gate CR_k changes the state by at most $\|CR_k - I\| = |e^{2\pi i/2^k} - 1| \leq 2\pi/2^k$, and errors add, so discarding all rotations with $k > k_{\max}$ on all n qubits costs at most

$$n \sum_{k > k_{\max}} \frac{2\pi}{2^k} < \frac{2\pi n}{2^{k_{\max}}}. \quad (25)$$

To make this smaller than ϵ it suffices to take $k_{\max} = \log_2(2\pi n/\epsilon)$. For an error that is polynomially small in n this is $k_{\max} = \mathcal{O}(\log n)$, giving an *approximate* QFT of $\mathcal{O}(n \log n)$ gates.

This is not a curiosity. On a fault-tolerant machine every rotation must itself be synthesised from a discrete gate set, at a cost logarithmic in the required precision, so rotations by absurdly

small angles are expensive rather than free. Every serious resource estimate for Shor's algorithm, including those quoted in Lecture 9, Block 9, uses the approximate transform.

3.7 Advantage of QFT

We can now answer the question this lecture has been circling. Can a quantum computer be used to compute discrete Fourier transforms quickly?

No. The circuit of Section 3.4 transforms a quantum *state*. To Fourier-transform a list of $N = 2^n$ data values one must first prepare the state $|f\rangle = \sum_j f_j |j\rangle$, and for generic data that requires specifying 2^n amplitudes, which takes exponentially many gates. Even if the state were somehow given, the output is a state, and measuring it does not reveal the amplitudes $\hat{f}_k$: it returns a single index k with probability $|\hat{f}_k|^2$, and learning the whole spectrum would require exponentially many repetitions. The exponential advantage in (24) is entirely consumed at the input and the output.

Yes. There is a class of problems for which neither bottleneck applies: those in which the input state is not arbitrary data but something a short circuit can prepare, and the desired output is not the whole spectrum but a single index, the location of a spike. For those, the $\mathcal{O}(n^2)$ circuit is pure profit, and the QFT is a *subroutine* inside a larger algorithm rather than an algorithm in itself.

The rule to remember. The quantum Fourier transform is efficient, and that fact is useless by itself. It becomes useful only in an algorithm with cheap read-in and cheap read-out: a state that can be prepared by a short circuit, and an answer that is a single measurement outcome. Every claim of a quantum speed-up should be interrogated on exactly these two points, and most exaggerated claims fail on one of them.

3.8 Two Reasons It Is Not Yet Practical

The gate count (24) hides two obstacles that decide whether the circuit can actually be run, and both are worth stating before Lecture 8 starts using the transform as though it were free.

The rotations are too small to apply. The gate R_k turns the state by an angle $2\pi/2^k$, and the circuit needs every k from 2 up to n . Already at $k = 20$ that angle is about 6×10^{-6} radians, far below the accuracy to which any physical gate can be calibrated, which is a part in 10^3 or 10^4 on current hardware. At the $n = 2048$ of Lecture 9 the smallest rotation is $2\pi/2^{2048}$, a number with no physical meaning whatever. On a fault-tolerant machine the situation is different but no easier: there an arbitrary rotation is not a gate at all, but a sequence of roughly $\log_2(1/\epsilon)$ gates from a discrete set, synthesised to precision ϵ , so small rotations are the *expensive* ones. Either way the exact circuit of Figure 3 is a fiction, and what one actually builds is the approximate transform of Section 3.6, which discards precisely the rotations that cannot be trusted. That is the real reason the approximate QFT matters.

The circuit assumes every qubit can talk to every other. Look again at Figure 3. Qubit 1 takes a controlled rotation from qubit 2 and from qubit 3; in general, qubit 1 interacts with all $n - 1$ others, qubit 2 with $n - 2$ of them, and so on. The circuit therefore presumes *all-to-all connectivity*: any pair of qubits may be acted on by a two-qubit gate directly.

Very little hardware provides this. A superconducting processor couples each qubit to two or three fixed neighbours on a planar lattice, and a gate between distant qubits must be routed by a chain of SWAP gates, each of which is itself three CNOTs and each of which adds error. Compiling an all-to-all circuit onto a two-dimensional grid of n qubits typically costs a further

factor of order $\sqrt{n}$ in gate count and depth. Trapped ions and neutral atoms come much closer to all-to-all connectivity, which is one of their principal advantages and one reason the comparison table of Lecture 2 listed connectivity as a figure of merit in its own right.

Taken together with the coherence requirement that Lecture 8, Section 6.4 will impose, this is why the quantum Fourier transform, for all that it is the best-understood subroutine in the subject, has never been run at a size anyone could use.

Looking ahead. Which problems have cheap read-in and cheap read-out? The next lecture identifies the canonical one. If a unitary U has an eigenstate $|u\rangle$ we can prepare, then the eigenvalue $e^{2\pi i\varphi}$ is a single number, and phase kickback writes φ into a register in exactly the Fourier-transformed form that $\text{QFT}^\dagger$ knows how to undo. Reading a phase turns out to be the one thing a quantum computer does supremely well, and Lecture 9 will show that an innocuous question about remainders can be rewritten as the reading of a phase.

Part II

Lecture 8: Phase Estimation

Lecture 7 ended with a warning and a promise. The warning: the quantum Fourier transform is efficient, and that fact alone is worthless, because preparing an arbitrary input state and reading out an arbitrary output state are both exponentially expensive. The promise: there exists a family of problems for which neither bottleneck applies, and for which the transform is therefore pure profit.

This lecture constructs that family. Its members all have the following shape. We are given a unitary U , either as a circuit or as a physical process we can run, together with an eigenstate $|u\rangle$. Since U is unitary its eigenvalues are pure phases, so

$$U|u\rangle = e^{2\pi i\varphi}|u\rangle, \quad \varphi \in [0, 1), \quad (26)$$

and the question is: what is φ ?

The input is cheap by assumption, since we were handed $|u\rangle$. The output is cheap because it is a single real number rather than 2^n amplitudes. Both of Lecture 7's objections are answered, and what remains is to build the algorithm, which will turn out to be nothing but phase kickback followed by an inverse Fourier transform.

Phase estimation is among the most important subroutines in quantum computing. Most algorithms with a claim to practical relevance contain it: Shor's factoring algorithm, which is Lecture 9; the estimation of molecular ground-state energies, where the energy is literally an eigenvalue; quantum counting and amplitude estimation, which run phase estimation on the Grover operator; and the quantum linear-systems algorithm, which uses it to invert the eigenvalues of a matrix.⁴ It is worth knowing well.

4 Block 4: Reading a Phase Off a Register

Learning goals:

- State the phase estimation problem and the two resources it assumes.
- Analyse the Hadamard test and explain why repeated sampling costs $\mathcal{O}(1/\epsilon^2)$.
- Compute the state of the counting register after controlled applications of U^{2^j} .
- Recognise that state as the quantum Fourier transform of the answer, and hence read the answer off with QFT[†].
- Draw the phase estimation circuit and count its resources.

A short raison d'être

The obvious way uses a single ancilla qubit and an interference experiment: put the ancilla in $|+\rangle$, apply U conditionally on it, and interfere. The probability of the outcome 0 is then a known function of φ , so repeating the experiment many times and counting gives an estimate. This works, and it is the standard way to measure a phase in a laboratory.

⁴All four are treated in Section 6 or in Lecture 9. One qualification: amplitude estimation *as originally formulated* is phase estimation, and that is the version described here, but several later variants reach the same accuracy without a Fourier transform and with much shallower circuits, precisely to avoid the cost catalogued in Section 6.4.

Its defect is statistical: estimating a probability to precision ϵ needs $\mathcal{O}(1/\epsilon^2)$ repetitions, by the same square-root law that governs any counting experiment.

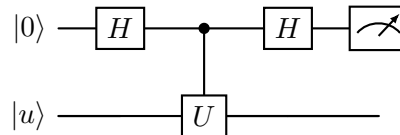
Phase estimation replaces the t independent one-qubit experiments with one coherent t -qubit experiment, and it achieves precision 2^{-t} with a single run. The mechanism is that the t ancillas do not each measure φ independently; they measure $\varphi, 2\varphi, 4\varphi, \dots, 2^{t-1}\varphi$, and those numbers are the successive binary digits of the answer. Getting them out requires undoing a Fourier transform, and Lecture 7 built the circuit that does it.

4.1 The Problem and its Assumptions

1. *We can prepare the eigenstate $|u\rangle$.* This is a strong assumption. If U is given only as a circuit then its eigenstates are generally as hard to find as its eigenvalues. Section 5.4 shows what happens when the assumption fails, and the answer turns out to be surprisingly benign.
2. *We can apply the controlled operations $C\text{-}U^{2^j}$ for $j = 0, 1, \dots, t-1$.* Applying U^{2^j} by brute force means 2^j applications of U , which is exponentially many; the assumption is really that the structure of U lets us do better. In Lecture 9 it will, because U there is multiplication by a constant and the constant can be squared classically in advance.

Why are the eigenvalues phases at all? Because U is unitary: by Lecture 4, Block 3, a unitary operator is normal, hence diagonalisable in an orthonormal eigenbasis, and its eigenvalues have unit modulus, since $\|U|u\rangle\| = \||u\rangle\|$ forces $|\lambda| = 1$. Writing $\lambda = e^{2\pi i\varphi}$ with $\varphi \in [0, 1)$ is then merely a choice of parametrisation, one that puts the factor 2π where Lecture 7's binary fractions want it.

4.2 The Obvious Method: the Hadamard Test



Trace the state through. After the first Hadamard we have $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|u\rangle$. The controlled- U leaves the $|0\rangle$ branch alone and multiplies the $|1\rangle$ branch by U , which, because $|u\rangle$ is an eigenstate, simply attaches the number $e^{2\pi i\varphi}$:

$$\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i\varphi}|1\rangle) \otimes |u\rangle. \quad (27)$$

This is *phase kickback*. The gate was applied to the second register and the second register did not change; the entire effect of the operation appears as a relative phase on the *control*. Lecture 5 saw the same phenomenon for a CNOT acting on $|+-\rangle$, and Lecture 7, Section 1.2 used it to convert a Boolean oracle into a phase oracle. Here it is the whole mechanism: the eigenvalue has been transferred from a place we cannot look at to a place we can.

The final Hadamard converts the relative phase into a probability:

$$H \frac{|0\rangle + e^{2\pi i\varphi}|1\rangle}{\sqrt{2}} = \frac{1 + e^{2\pi i\varphi}}{2}|0\rangle + \frac{1 - e^{2\pi i\varphi}}{2}|1\rangle,$$

so that

$$p(0) = \left| \frac{1 + e^{2\pi i\varphi}}{2} \right|^2 = \cos^2(\pi\varphi) = \frac{1 + \cos(2\pi\varphi)}{2}. \quad (28)$$

So each run returns one bit, and the bit is 1 with a probability that depends on φ . Estimating that probability to precision ϵ requires, by the law of large numbers,

$$M = \mathcal{O}(1/\epsilon^2) \quad (29)$$

repetitions: the standard error of a proportion estimated from M samples falls only as $1/\sqrt{M}$. Lecture 2 met the same $1/\sqrt{M}$ in the context of readout statistics; it is not a quantum effect but an ordinary fact about sampling.

Equation (29) is the benchmark. To gain one more bit of precision one must quadruple the number of runs. An algorithm that instead gains a bit per *qubit* would be exponentially better, and that is what we now build.

4.3 Controlled Powers and the Binary Expansion

Repeat the circuit of Section 4.2, but with U^{2^k} in place of U . Since $U^{2^k} |u\rangle = e^{2\pi i 2^k \varphi} |u\rangle$, equation (27) becomes

$$\frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 2^k \varphi} |1\rangle) \otimes |u\rangle. \quad (30)$$

The ancilla now carries the phase $2^k \varphi$ rather than φ : the higher the power, the faster the phase winds, and the more sensitive the ancilla is to small changes in φ .

Now recall the convention of Lecture 7, Section 3.2, and suppose for the moment that φ has an exact t -bit binary expansion,

$$\varphi = 0.\varphi_1\varphi_2\cdots\varphi_t = \frac{\varphi_1}{2} + \frac{\varphi_2}{4} + \cdots + \frac{\varphi_t}{2^t}. \quad (31)$$

Multiplying by 2^k shifts the binary point k places to the right,

$$2^k \varphi = \underbrace{\varphi_1 \cdots \varphi_k}_{\text{an integer}} \cdot \varphi_{k+1} \cdots \varphi_t,$$

and since $e^{2\pi i M} = 1$ for any integer M , only the fractional part survives in (30):

$$e^{2\pi i 2^k \varphi} = e^{2\pi i 0.\varphi_{k+1}\cdots\varphi_t}. \quad (32)$$

So the ancilla controlling U^{2^k} ends up carrying a phase built from the *last* $t - k$ bits of φ . The ancilla controlling $U^{2^{t-1}}$ carries $0.\varphi_t$, a single bit and hence a sign, while the ancilla controlling $U^{2^0} = U$ carries the whole of $0.\varphi_1\cdots\varphi_t$.

4.4 The Exact Case

Take t ancillas, all in $|+\rangle$, and let ancilla l (counted from the top, as in Lecture 7) control $U^{2^{t-l}}$. By (32) with $k = t - l$, the counting register ends in the product state

$$\bigotimes_{l=1}^t \frac{|0\rangle + e^{2\pi i 0.\varphi_{t-l+1}\cdots\varphi_t} |1\rangle}{\sqrt{2}}. \quad (33)$$

Compare this with Theorem 3.2 of Lecture 7. It is the same expression, character for character, with j_m replaced by φ_m . Therefore:

Theorem 4.1 (the counting register holds a Fourier transform). *If $\varphi = 0.\varphi_1\cdots\varphi_t$ exactly, then after the controlled operations the counting register is in the state*

$$\text{QFT } |\Phi\rangle, \quad \Phi = 2^t \varphi = \varphi_1\varphi_2\cdots\varphi_t \text{ in binary,} \quad (34)$$

an integer with $0 \leq \Phi < 2^t$.

The consequence is immediate and is the whole algorithm. Apply $\text{QFT}^\dagger$, which Lecture 7, Section 3.5 built from $\mathcal{O}(t^2)$ gates. The register becomes $|\Phi\rangle$ exactly. Measure every qubit in the computational basis. The outcome is the bit string $\varphi_1\varphi_2\cdots\varphi_t$, which *is* the binary expansion of φ , obtained with certainty in a single run.

Example 4.2 ($t = 3$, $\varphi = 5/8$). Here $\varphi = 0.101$ in binary, so $\Phi = 5$. The three ancillas control U^4 , U^2 and U^1 respectively and acquire the phases

$$e^{2\pi i 0.1} = e^{i\pi} = -1, \quad e^{2\pi i 0.01} = e^{i\pi/2} = i, \quad e^{2\pi i 0.101} = e^{i5\pi/4},$$

so the counting register is

$$\frac{|0\rangle - |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + i|1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{5i\pi/4}|1\rangle}{\sqrt{2}}.$$

Expanding the product gives $\frac{1}{2\sqrt{2}} \sum_{k=0}^7 e^{2\pi i \cdot 5k/8} |k\rangle$, which is $\text{QFT}|5\rangle$ by Definition 3.1. Applying $\text{QFT}^\dagger$ returns $|5\rangle = |101\rangle$, and the measurement reads off $\varphi = 0.101$.

4.5 The Circuit and its Cost

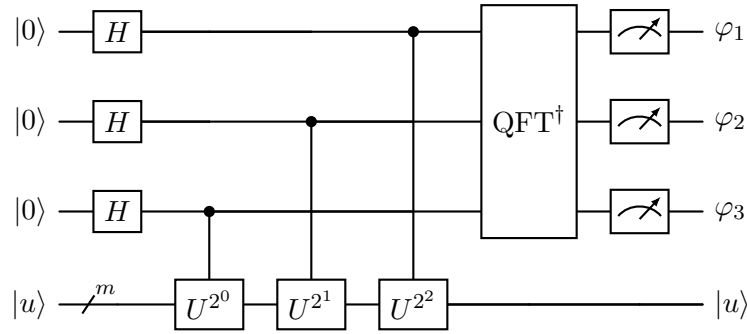


Figure 4: Phase estimation, drawn for $t = 3$; the general case adds counting qubits at the top, controlling successively higher powers. The bottom wire is the m -qubit register carrying the eigenstate, which is returned unchanged. Ancilla l , counted from the top, controls $U^{2^{t-l}}$, so the topmost ancilla controls the highest power. The inverse Fourier transform converts the product state (33) into the binary expansion of φ , most significant bit at the top.

Resource summary for phase estimation.

Resource	Count
counting qubits	t , one per bit of output precision
eigenstate register	m , the size of the register U acts on
Hadamards	t
gates for $\text{QFT}^\dagger$	$\mathcal{O}(t^2)$, or $\mathcal{O}(t \log t)$ for the approximate transform
applications of U	$\sum_{j=0}^{t-1} 2^j = 2^t - 1$

The last line is the one that matters, and it is easy to overlook. The *gate* count of the Fourier transform is quadratic and cheap; the number of applications of U is $2^t - 1$, which is exponential in t . Phase estimation is efficient only when U^{2^j} can be implemented at a cost polynomial in j rather than by 2^j repetitions. Section 6.4 returns to this, and Lecture 9, Block 8 shows how order finding satisfies it.

Looking ahead. Everything so far assumed that φ has an exact t -bit expansion, which is false for almost every real number, and that we know t in advance, which is usually false too. The next block removes both assumptions, at the cost of replacing certainty by a probability, which we will then have to bound.

5 Block 5: When the Phase is not Exact

Learning goals:

- Compute the output distribution of phase estimation for an arbitrary phase.
- Prove that the best t -bit estimate is returned with probability at least $4/\pi^2$.
- State how many counting qubits are needed for n correct bits with confidence $1 - \epsilon$, and compare the cost with the Hadamard test.
- Explain what happens when the input is a superposition of eigenstates rather than a single one.
- Describe iterative phase estimation and say why it matters for near-term hardware.

A short raison d'être

A general φ is irrational, so no finite register can hold it and the clean statement of Theorem 4.1 cannot survive. What survives is better than one might fear. The measurement no longer returns the right answer with certainty; it returns a distribution sharply peaked on the nearest t -bit value, with tails that fall off like the square of the distance. This is exactly the leakage of Lecture 7, Example 2.5, and for the same reason: a period that does not fit the register smears the peak without destroying it.

The quantitative statement is Theorem 5.1, and its constant $4/\pi^2$ will be used by name in Lecture 9.

5.1 The Output Distribution

Drop the assumption (31). Equation (30) is unaffected, so after the controlled operations the counting register is in the state

$$|\Psi\rangle = \bigotimes_{l=1}^t \frac{|0\rangle + e^{2\pi i 2^{t-l}\varphi} |1\rangle}{\sqrt{2}} = \frac{1}{2^{t/2}} \sum_{z=0}^{2^t-1} e^{2\pi i \varphi z} |z\rangle, \quad (35)$$

where the second form follows by expanding the product exactly as in the proof of Theorem 3.2, read backwards.

Apply $\text{QFT}^\dagger$, whose matrix elements are $2^{-t/2} e^{-2\pi i z \ell / 2^t}$, and read off the amplitude of the outcome ℓ :

$$\alpha_\ell = \frac{1}{2^t} \sum_{z=0}^{2^t-1} e^{2\pi i z (\varphi - \ell/2^t)} = \frac{1}{2^t} \cdot \frac{1 - e^{2\pi i 2^t \delta_\ell}}{1 - e^{2\pi i \delta_\ell}}, \quad \delta_\ell := \varphi - \frac{\ell}{2^t}, \quad (36)$$

the sum being geometric with ratio $e^{2\pi i \delta_\ell}$. Taking the modulus squared,

$$p(\ell) = |\alpha_\ell|^2 = \frac{1}{2^{2t}} \frac{\sin^2(\pi 2^t \delta_\ell)}{\sin^2(\pi \delta_\ell)}. \quad (37)$$

If $\delta_\ell = 0$, which is the exact case of Block 4, both numerator and denominator vanish and the limit is $p(\ell) = 1$, recovering Theorem 4.1. Otherwise (37) is a sharply peaked function of ℓ , the discrete analogue of a diffraction pattern, and the same expression met in Lecture 7 when the period failed to divide the register size.

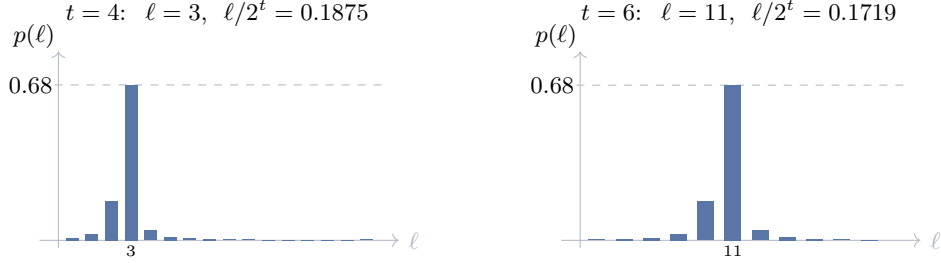


Figure 5: The output distribution (37) for $\varphi = 1/6$, which has no finite binary expansion. Left: $t = 4$ counting qubits; right: $t = 6$, with only the window $6 \leq \ell \leq 16$ drawn. The peak sits on the nearest representable value and carries probability 0.68 in both cases, comfortably above the guaranteed $4/\pi^2 = 0.405$ of Theorem 5.1, and, as that theorem promises, *independent of t* . What extra qubits buy is not a taller peak but a finer grid: $\ell/2^t$ moves from 0.1875 to 0.1719, closing in on $0.1\bar{6}$.

5.2 The $4/\pi^2$ Bound

Theorem 5.1 (phase estimation returns the best estimate with constant probability). *Let b be the integer such that $b/2^t$ is the closest t -bit value to φ , so that $|\delta_b| \leq 2^{-(t+1)}$. Then*

$$p(b) \geq \frac{4}{\pi^2} \approx 0.405. \quad (38)$$

Proof. Assume $\delta_b \neq 0$; the case $\delta_b = 0$ gives $p(b) = 1$. We bound the numerator and the denominator of (37) separately.

For the numerator, use the concavity of the sine on $[0, \pi]$, which gives the elementary inequality $|\sin(\pi x)| \geq 2|x|$ for $|x| \leq \frac{1}{2}$. Since $|2^t \delta_b| \leq \frac{1}{2}$ by the choice of b , this applies with $x = 2^t \delta_b$:

$$\sin^2(\pi 2^t \delta_b) \geq 4 \cdot 2^{2t} \delta_b^2.$$

For the denominator, use $|\sin \theta| \leq |\theta|$ with $\theta = \pi \delta_b$:

$$\sin^2(\pi \delta_b) \leq \pi^2 \delta_b^2.$$

Substituting both into (37),

$$p(b) \geq \frac{1}{2^{2t}} \cdot \frac{4 \cdot 2^{2t} \delta_b^2}{\pi^2 \delta_b^2} = \frac{4}{\pi^2},$$

and the factors of δ_b^2 and 2^{2t} cancel completely, so the bound is independent of both φ and t . $\square$

The bound is tight: it is approached when φ sits exactly halfway between two t -bit values, which is the worst case, and it improves to 1 as φ approaches a representable value. A probability of 0.405 is perfectly workable given the ability to check an answer, which Lecture 9 will have.

5.3 How Many Counting Qubits

Theorem 5.1 concerns the single most likely outcome. In practice one wants a weaker and more useful guarantee: that the first n bits of the answer are right, allowing the outcome to be any of several nearby values.

Theorem 5.2 (register size). *To obtain an estimate of φ accurate to n bits with probability of success at least $1 - \epsilon$, it suffices to use a counting register of*

$$t = n + \left\lceil \log_2 \left(2 + \frac{1}{2\epsilon} \right) \right\rceil \quad (39)$$

qubits.

Proof. Write $t = n + p$, let b be the best t -bit estimate as in Theorem 5.1, and label the outcomes by their displacement from it, $\ell = b + j$ with j taken modulo 2^t in the range $-2^{t-1} < j \leq 2^{t-1}$. Getting the leading n bits right is implied by $|j| \leq e$ with $e := 2^p - 1$, so we must bound the probability that $|j| > e$.

Step 1: an amplitude bound. From (36), the amplitude of the outcome $b + j$ is

$$\alpha_{b+j} = \frac{1}{2^t} \cdot \frac{1 - e^{2\pi i(2^t \delta_b - j)}}{1 - e^{2\pi i(\delta_b - j/2^t)}}.$$

The numerator has modulus at most 2. For the denominator we use the elementary inequality $|1 - e^{i\theta}| \geq 2|\theta|/\pi$, valid for $|\theta| \leq \pi$, which is the same concavity of the sine used in Theorem 5.1; with $\theta = 2\pi(\delta_b - j/2^t)$ it gives $|1 - e^{i\theta}| \geq 4|\delta_b - j/2^t|$. Hence

$$|\alpha_{b+j}| \leq \frac{1}{2^{t+1} |\delta_b - j/2^t|} = \frac{1}{2 |2^t \delta_b - j|}. \quad (40)$$

Step 2: use the definition of b . By construction $|2^t \delta_b| \leq \frac{1}{2}$, so for $|j| \geq 1$ we have $|2^t \delta_b - j| \geq |j| - \frac{1}{2}$, and therefore

$$p(b + j) = |\alpha_{b+j}|^2 \leq \frac{1}{4(|j| - \frac{1}{2})^2}.$$

Step 3: sum the two tails. Summing over $|j| > e$ and comparing the sum with an integral,

$$\Pr[|j| > e] \leq 2 \sum_{j=e+1}^{\infty} \frac{1}{4(j - \frac{1}{2})^2} \leq \frac{1}{2} \int_{e-1}^{\infty} \frac{du}{u^2} = \frac{1}{2(e-1)}.$$

Step 4: choose p . Requiring $1/(2(e-1)) \leq \epsilon$ gives $e \geq 1 + 1/(2\epsilon)$, that is $2^p \geq 2 + 1/(2\epsilon)$, which is (39). $\square$

The bound is generous: the numerical experiment of Figure 5 shows the true tail to be an order of magnitude smaller than Step 3 allows. What matters is its shape, not its constant.

The shape of (39) is worth stating in words: **the number of qubits is the number of desired bits, plus a constant that depends only logarithmically on the confidence required.** Doubling the precision costs one more qubit. Improving the confidence from 99% to 99.9% costs about three more qubits in total.

Now compare with the Hadamard test. To reach precision $\epsilon = 2^{-n}$ that method needed $\mathcal{O}(1/\epsilon^2) = \mathcal{O}(4^n)$ repetitions, by (29). Phase estimation reaches the same precision with $n + \mathcal{O}(1)$ qubits and $\mathcal{O}(2^n)$ applications of U in a single coherent run. That is a quadratic improvement in the total number of uses of U , and an exponential improvement in the number of *experiments*. This quadratic gain is the real content of the algorithm, and it is frequently misdescribed as exponential; it is exponential only relative to the number of qubits, which is not a cost one pays in the same currency.

5.4 Superposed Eigenstates

We now abandon the first assumption of Section 4.1. Suppose we cannot prepare a single eigenstate, but can prepare some state $|\psi\rangle$ which, expanded in the eigenbasis of U , reads

$$|\psi\rangle = \sum_u c_u |u\rangle, \quad U|u\rangle = e^{2\pi i \varphi_u} |u\rangle. \quad (41)$$

The circuit of Figure 4 is linear, so it acts on each term separately. Writing $\widetilde{\varphi}_u$ for the t -bit estimate the circuit would produce from $|u\rangle$ alone, linearity gives the output

$$\sum_u c_u |\widetilde{\varphi}_u\rangle_c \otimes |u\rangle_w, \quad (42)$$

an entangled state of the two registers. Measuring the counting register therefore returns an estimate of φ_u , with probability $|c_u|^2$ by the Born rule, and collapses the second register into the corresponding $|u\rangle$.

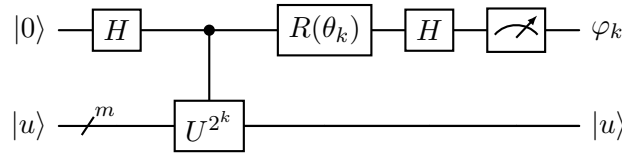
Phase estimation on a superposition samples the spectrum. Feeding $\sum_u c_u |u\rangle$ returns a random eigenvalue, drawn with probability $|c_u|^2$, and leaves the work register in the matching eigenstate. We do not learn *which* eigenstate we obtained. Read pessimistically, this says the algorithm degrades to a sampling procedure when the eigenstate cannot be prepared. Read optimistically, which is how Lecture 9 will read it, it says that the eigenstate need not be prepared at all, provided a random eigenvalue is as good as a chosen one. Order finding is precisely a problem where it is: every eigenvalue of the relevant operator contains the answer.

5.5 Iterative Phase Estimation

The circuit of Figure 4 needs t ancillas held coherently while $2^t - 1$ applications of U run. On present hardware that is the binding constraint. There is a variant that removes it almost entirely.

The observation is that the inverse Fourier transform of Lecture 7 consists of Hadamards and *controlled* phase rotations, and that every control acts on a qubit which is about to be measured. A controlled operation whose control is measured immediately afterwards may instead be performed classically: measure the control first, and apply the rotation only if the outcome was 1. This is the *semiclassical* Fourier transform.

Making that replacement everywhere leaves a circuit that uses a single ancilla, reused t times:



Run this for $k = t - 1, t - 2, \dots, 0$ in turn, extracting one bit of φ per round, starting from the least significant. The angle θ_k is computed classically from the bits already measured, and it undoes exactly the phase contributions those bits are responsible for.

The trade is qubits and coherence time against classical feedback and repetition: the total number of applications of U is unchanged, but only $m + 1$ qubits are ever needed, and the ancilla is measured and reset between rounds so that it need only stay coherent for one round at a time. This is the version, often called Kitaev's algorithm, that near-term experiments actually

run, and the reason that the phrase “phase estimation was demonstrated on hardware” should always prompt the question “which version?”.

Looking ahead. We have an instrument for reading eigenvalues. The next block asks what one might want to point it at, and the following lecture answers that question in the way that made the subject famous.

6 Block 6: What Phase Estimation is For

Learning goals:

- Explain how phase estimation yields the eigenvalues of a Hamiltonian, and state the limitation imposed by state preparation.
- Describe the Grover operator, identify its eigenvalues, and show that phase estimation on it performs amplitude estimation.
- State the cost that makes phase estimation a fault-tolerant-era algorithm.

A short raison d’être

An algorithm that reads eigenvalues sounds like a specialised tool. It is in fact the opposite, for a simple reason: an enormous number of quantities that people want to compute are eigenvalues of something. The energy of a molecule is an eigenvalue of its Hamiltonian. The number of solutions to a search problem is, as we are about to see, encoded in an eigenvalue of the Grover operator. And in Lecture 9, the order of an integer modulo N will be encoded in an eigenvalue of a multiplication operator.

This block surveys the first two and closes with the cost, which is the part usually omitted.

6.1 Energies

Lecture 6 established the original motivation for quantum computing: a quantum computer can simulate the time evolution of a quantum system efficiently, because that evolution is a unitary $U = e^{-iH\tau/\hbar}$ and Trotter–Suzuki methods build it from a polynomial number of gates. Here τ is the simulated time; we write it as τ rather than t because in this lecture t already denotes the number of counting qubits, and the two have nothing to do with each other.

Suppose we do so, and take $|u\rangle$ to be an energy eigenstate, $H|u\rangle = E|u\rangle$. Then, by the operator-function machinery of Lecture 4, Block 4,

$$U|u\rangle = e^{-iE\tau/\hbar}|u\rangle, \quad (43)$$

an eigenvalue equation of exactly the form (26) with $\varphi = -E\tau/(2\pi\hbar)$. Phase estimation therefore measures energies, and to n bits of precision with $n + \mathcal{O}(1)$ ancillas. The simulated time τ is chosen so that the phase lands in $[0, 1)$, which requires only a rough prior bound on E , generally available.

The application that drives most investment in the subject is the electronic ground-state energy of a molecule, which determines reaction rates and binding affinities and which classical methods compute only approximately for strongly correlated systems.

There is a caveat, and it is the one from Section 5.4. We cannot prepare the ground state $|u_0\rangle$; if we could, we would probably already know its energy. What we can prepare is a trial

state $|\psi\rangle$ believed to have a large overlap with it. By (42), phase estimation then returns the ground-state energy with probability $|\langle u_0|\psi\rangle|^2$ and some other eigenvalue otherwise, so the number of repetitions scales as $1/|\langle u_0|\psi\rangle|^2$. For large molecules that overlap is believed to shrink exponentially with system size for generic trial states, and constructing good trial states is consequently an active research problem rather than a solved one. The quantum speed-up is real, and it is contingent on a classical-sounding question about initial guesses.

6.2 Amplitudes, and Where Grover Fits

The second application also explains the place, in this course, of the other famous quantum algorithm.

Let $f : \{0,1\}^n \rightarrow \{0,1\}$ mark a set of M solutions among $N = 2^n$ candidates, and let $U_f^\pm$ be its phase oracle as in Lecture 7, Section 1.2. Write $|s\rangle = H^{\otimes n} |0\rangle^{\otimes n}$ for the uniform superposition, and split it into its solution and non-solution parts,

$$|s\rangle = \sin \theta |w\rangle + \cos \theta |w^\perp\rangle, \quad \sin \theta = \sqrt{\frac{M}{N}}, \quad (44)$$

where $|w\rangle$ is the normalised uniform superposition of solutions and $|w^\perp\rangle$ that of non-solutions. Define the *Grover operator*

$$G = U_s U_f^\pm, \quad U_s = 2|s\rangle\langle s| - I. \quad (45)$$

Both factors are reflections: $U_f^\pm$ reflects about $|w^\perp\rangle$, since it flips the sign of the solution component and leaves the rest alone, and U_s reflects about $|s\rangle$. A composition of two reflections in a plane is a rotation by twice the angle between their axes, so G acts on the two-dimensional real span of $|w\rangle$ and $|w^\perp\rangle$ as a rotation by 2θ towards $|w\rangle$.

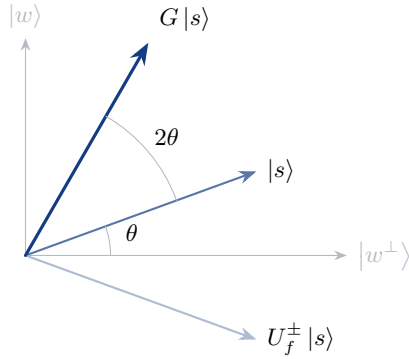


Figure 6: The Grover operator as a rotation. The oracle reflects $|s\rangle$ about the non-solution axis $|w^\perp\rangle$; the diffuser U_s reflects the result about $|s\rangle$. The composition is a rotation by 2θ towards the solution axis $|w\rangle$, and it is independent of which states are marked.

Search, in one paragraph. Since G rotates by 2θ each time, applying it k times to $|s\rangle$ gives $\sin((2k+1)\theta) |w\rangle + \cos((2k+1)\theta) |w^\perp\rangle$. Choosing k so that $(2k+1)\theta \approx \pi/2$ aligns the state with $|w\rangle$, and a measurement then returns a solution with probability close to one. For $M \ll N$ we have $\theta \approx \sqrt{M/N}$, so

$$k \approx \frac{\pi}{4} \sqrt{\frac{N}{M}}, \quad (46)$$

against the $\mathcal{O}(N/M)$ queries an unstructured classical search requires. This is Grover's algorithm, and the speed-up is *quadratic*, not exponential. Note also that overshooting is possible: the rotation continues past $|w\rangle$ if k is too large, so more iterations are not better.

Counting, which is phase estimation. A rotation by 2θ in a plane has eigenvalues $e^{\pm 2i\theta}$, with eigenvectors the two complex combinations $(|w\rangle \pm i|w^\perp\rangle)/\sqrt{2}$. And $|s\rangle$, which we can prepare with n Hadamards, is a superposition of exactly those two eigenvectors. So run phase estimation on G with input $|s\rangle$: by Section 5.4 it returns $\pm\theta/\pi$, from which

$$M = N \sin^2 \theta \quad (47)$$

follows immediately. This is *quantum counting*: it determines the number of solutions, to n bits of precision, using $\mathcal{O}(2^n)$ oracle calls, and in particular it decides whether any solution exists at all, which unstructured classical search cannot do without exhausting the space.

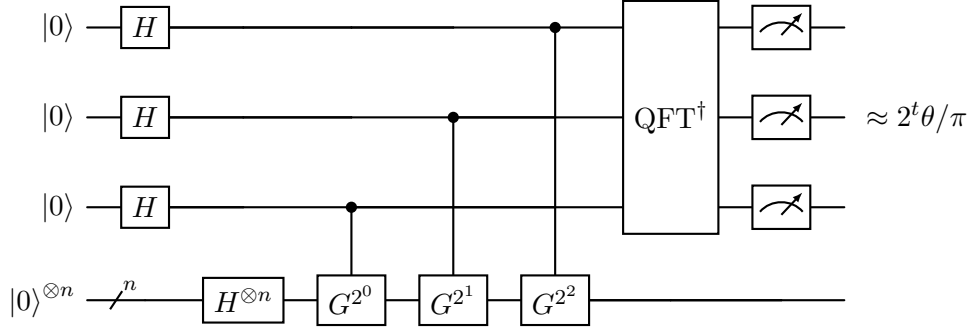


Figure 7: Quantum counting, drawn for $t = 3$ counting qubits. It is exactly the phase estimation circuit of Figure 4 with U replaced by the Grover operator G and the eigenstate replaced by $|s\rangle = H^{\otimes n} |0\rangle^{\otimes n}$, which the $H^{\otimes n}$ layer on the bottom wire prepares. The measured integer estimates $2^t \theta / \pi$, and (47) converts it into the number of solutions M . No oracle is needed beyond the one already inside G .

The generalisation is *amplitude estimation*. Replace $H^{\otimes n}$ by any circuit A that prepares a state $|\psi\rangle = A |0\rangle^{\otimes n}$, and suppose the possible measurement outcomes have been divided into *good* ones and *bad* ones, with Π the projector onto the good ones. The quantity to be estimated is the probability of a good outcome,

$$a = \|\Pi A |0\rangle^{\otimes n}\|^2, \quad (48)$$

which in the counting problem above is just $a = M/N = \sin^2 \theta$. Building the Grover operator out of A and Π in place of $H^{\otimes n}$ and $U_f^\pm$, and running phase estimation on it, yields a to precision ϵ with $\mathcal{O}(1/\epsilon)$ uses of A , against the $\mathcal{O}(1/\epsilon^2)$ that ordinary repeated sampling of A would require. This quadratic gain applies to Monte Carlo integration, and hence to derivative pricing and risk calculation, which is why it is the most-studied quantum algorithm in finance.

A caveat on quadratic speed-ups, worth carrying to Lecture 10. A quadratic speed-up is a fragile thing. The classical cost is $\mathcal{O}(N)$ and the ideal quantum cost $\mathcal{O}(\sqrt{N})$, but the quantum cost must be multiplied by the overhead of making the computation reliable. That overhead is either error mitigation, which Lecture 10 will show costs a factor growing polynomially in the circuit size, or full error correction, which costs a large constant factor and many more qubits. For plausible problem sizes that overhead can exceed the square root one has gained.

The conclusion is not that Grover is useless, but that the algorithms most likely to matter first are those where the classical cost is exponential and the quantum cost is low-degree polynomial, because only there is there room for a large overhead. Phase estimation applied to simulation, and the factoring algorithm of Lecture 9, are of that kind.

6.3 One-Line Mentions

For completeness, two further families in which phase estimation is the engine. The *quantum linear-systems algorithm* of Harrow, Hassidim and Lloyd uses phase estimation on $e^{iA\tau}$ to invert the eigenvalues of a matrix A and thereby prepare a state proportional to $A^{-1}|b\rangle$; it is efficient only for sparse, well-conditioned A , and it returns a *state*, not a vector, so Lecture 7's read-out objection applies in full force. *Quantum walk* algorithms extract the spectral gap of a walk operator by phase estimation, which is how the quadratic speed-ups for element distinctness and triangle finding are obtained.

6.4 What It Costs

We end with the number that any honest account must include.

The resource table of Section 4.5 records $2^t - 1$ applications of U . These are not parallel and they are not independent: they form a single coherent sequence, at the end of which the accumulated phases must still interfere correctly. So the *circuit depth* is $2^t - 1$ applications of U , and the entire sequence must complete within the coherence time of the device, with a total error small enough that the interference survives.

Lecture 2 put the relevant figure of merit at T_2/t_{gate} , the number of gates that fit inside the coherence time, and observed that today it is of order 10^3 to 10^5 . Phase estimation to n bits needs a coherent sequence of order 2^n applications of U , each of which is itself many gates. For the $n = 2048$ of Lecture 9 this is many orders of magnitude beyond any device that exists or is planned as a physical machine.

Phase estimation is a fault-tolerant-era algorithm. It is not a NISQ algorithm, and claims that it is should be treated with suspicion unless they refer to the iterative variant of Section 5.5 on very small instances. The variational methods designed for present-day hardware exist precisely because phase estimation is out of reach, and they buy that accessibility by giving up the guarantee of accuracy that Theorem 5.2 provides.

Looking ahead. Everything is now in place. We have a Fourier transform that costs $\mathcal{O}(n^2)$ gates, an algorithm that uses it to read an eigenvalue to n bits with $n + \mathcal{O}(1)$ qubits, and the knowledge that a superposition of eigenstates yields a random eigenvalue rather than nothing at all. The next lecture exhibits a unitary, multiplication modulo an integer N , whose eigenvalues are $e^{2\pi i s/r}$, where r is the quantity whose classical intractability secures the internet. Every obstacle that remains is one of the three we have just catalogued: building the controlled powers, preparing the eigenstate, and converting an approximate phase back into an exact integer.

Part III

Lecture 9: Period Finding, Order Finding and Shor's Algorithm

Two lectures of machinery are now in place. Lecture 7 built a Fourier transform costing $\mathcal{O}(n^2)$ gates and warned that it is useful only inside an algorithm with cheap read-in and cheap read-out. Lecture 8 supplied the one such algorithm we know how to build: phase estimation, which takes a unitary and an eigenstate and returns the eigenvalue's phase.

What remains is to exhibit a problem of independent interest whose answer is an eigenvalue. Such a problem exists, it is elementary, and it concerns remainders: given integers x and N , find the smallest r for which x^r leaves remainder 1 on division by N . This lecture poses that question, solves it, and then shows that solving it efficiently breaks the cryptosystem that protects most of the traffic on the internet.

The structure is deliberately lopsided. Block 7 is entirely classical: modular arithmetic, RSA, and the reduction of factoring to a question about periods. Block 8 does the quantum work, which is short, because Lecture 8 did most of it. Block 9 converts the measured phase back into an integer and assesses what has and has not been achieved.

7 Block 7: Modular Arithmetic, RSA, and the Reduction to Order Finding

Learning goals:

- Work confidently in $\mathbb{Z}_N$: units, the greatest common divisor, Euclid's algorithm, and the order of an element.
- State how RSA works and exactly which step factoring breaks.
- Prove that a non-trivial square root of 1 modulo N yields a factor of N .
- State the probabilistic theorem that makes a random choice of base work, and assemble the complete factoring algorithm.
- Run the whole algorithm by hand on $N = 15$, including a case that fails.

A short raison d'être

The division of labour in Shor's algorithm is worth stating at the outset, because it is the opposite of what the popular account suggests. Shor's algorithm is a classical algorithm containing a single quantum subroutine. The content of this block, the reduction of factoring to order finding, is ordinary number theory that predates quantum computing entirely; it attracted little attention because order finding was itself believed to be hard. The quantum computer contributes exactly one thing, the order of an integer modulo N , and every other step runs on a classical machine.

Two consequences follow. The first concerns where to look for quantum advantage: at subroutines within problems, rather than at problems as a whole. The second concerns accounting. Since the quantum contribution is a single well-defined subroutine, its cost can be stated exactly, and Block 8 does so.

7.1 Arithmetic Modulo N

Throughout this lecture N denotes the integer we wish to factor and

$$L = \lceil \log_2 N \rceil \quad (49)$$

is the number of bits needed to write it down. This is the input size, and “efficient” will mean polynomial in L , not in N , which is exponentially larger. A trial-division algorithm that tests every candidate factor up to $\sqrt{N}$ runs in time $\mathcal{O}(2^{L/2})$ and is therefore useless for the $L = 2048$ of practical cryptography.

A clash of notation, flagged once. In Lectures 7 and 8 the symbol N meant 2^n , the dimension of the Fourier transform. In this lecture N is the number to be factored, which is almost never a power of two. Where a Fourier transform appears we shall write its dimension as 2^t , with t the size of the counting register. The two are related only by the inequality $2^t \approx N^2$, established in Block 9.

Recall the ring $\mathbb{Z}_N = \{0, 1, \dots, N-1\}$ with addition and multiplication taken modulo N . An element x has a multiplicative inverse in $\mathbb{Z}_N$ if and only if $\gcd(x, N) = 1$, in which case x is called a *unit*; the units form a group under multiplication, written $\mathbb{Z}_N^*$.

Definition 7.1 (Euler’s totient). $\phi(N)$ is the number of integers in $\{1, 2, \dots, N\}$ coprime to N , that is, the number of units: $\phi(N) = |\mathbb{Z}_N^*|$. For a prime p every one of $1, \dots, p-1$ is coprime to p , so $\phi(p) = p-1$; and for a product of two distinct primes, the case RSA uses, $\phi(pq) = (p-1)(q-1)$.

Definition 7.2 (greatest common divisor and Euclid’s algorithm). $\gcd(a, b)$ is the largest integer dividing both a and b . It is computed by the observation that $\gcd(a, b) = \gcd(b, a \bmod b)$, iterated until the second argument is zero:

$$\gcd(2048, 341) : \quad 2048 = 6 \cdot 341 + 2, \quad 341 = 170 \cdot 2 + 1, \quad 2 = 2 \cdot 1 + 0, \quad \text{so} \quad \gcd(2048, 341) = 1.$$

Every two steps at least halve the smaller argument, so the algorithm terminates after $\mathcal{O}(L)$ divisions. Charging $\mathcal{O}(L^2)$ for each of them gives a crude bound of $\mathcal{O}(L^3)$; but the divisions are not all expensive, and totalling their true costs gives $\mathcal{O}(L^2)$ bit operations, which is the figure we shall use. It is very fast, it is more than two thousand years old, and Shor’s algorithm calls it twice.

Definition 7.3 (order). Let $\gcd(x, N) = 1$. The *order* of x modulo N , written $r = \text{ord}_N(x)$, is the least positive integer such that

$$x^r \equiv 1 \pmod{N}. \quad (50)$$

The order exists because $\mathbb{Z}_N^*$ is a finite group: the powers $x, x^2, x^3, \dots$ cannot all be distinct, and once two coincide the sequence closes into a cycle. Lagrange’s theorem gives $r \mid \phi(N)$, and in particular $r < N$.

The word “order” hides the word we actually care about, which is *period*. Suppose we know r . Then for any k ,

$$x^{k+r} = x^k x^r \equiv x^k \pmod{N},$$

so the sequence $1, x, x^2, x^3, \dots$ taken modulo N is periodic with period exactly r . This is the function whose period we shall find.

Example 7.4. Take $N = 15$ and $x = 7$. Then $7^1 = 7$, $7^2 = 49 \equiv 4$, $7^3 \equiv 28 \equiv 13$, $7^4 \equiv 91 \equiv 1$, so $\text{ord}_{15}(7) = 4$ and the sequence $1, 7, 4, 13, 1, 7, 4, 13, \dots$ repeats with period 4. Taking instead $x = 4$ gives $4^2 = 16 \equiv 1$, so $\text{ord}_{15}(4) = 2$.

No classical algorithm is known that computes $\text{ord}_N(x)$ in time polynomial in L , and it is widely believed that none exists. The naive method, computing $x, x^2, x^3, \dots$ until the answer is 1, takes r steps, and r may be as large as N , that is, exponential in L .

7.2 Why Anyone Cares: RSA

The reason factoring is interesting is that a great deal of infrastructure assumes it is hard. Here is the whole of RSA, which is enough to see exactly where the assumption enters.

Algorithm 7.5 (RSA). *Key generation.* Choose two large primes p and q and set $N = pq$. Compute $\phi(N) = (p-1)(q-1)$. Choose a public exponent e with $\gcd(e, \phi(N)) = 1$, and compute the private exponent d with $ed \equiv 1 \pmod{\phi(N)}$, which Euclid's algorithm does quickly. Publish (N, e) ; keep d secret and discard p and q .

Encryption. A message is an integer $M < N$. The ciphertext is $C = M^e \bmod N$.

Decryption. $C^d = M^{ed} \equiv M \bmod N$, by Euler's theorem.

Both exponentiations are performed by repeated squaring and cost $\mathcal{O}(L^3)$; the scheme is practical. Its security rests on the claim that an eavesdropper holding (N, e) and C cannot find M . The obvious attack is to compute d , which requires $\phi(N) = (p-1)(q-1)$, which requires p and q . **Factor N and the private key falls out immediately.**

Classically that is currently out of reach. The best known general algorithm, the general number field sieve, runs in time

$$\exp\left(c L^{1/3} (\log L)^{2/3}\right) \quad \text{for a constant } c \approx 1.9, \quad (51)$$

which is sub-exponential but firmly super-polynomial. The largest RSA modulus factored to date is RSA-250, an 829-bit number, broken in February 2020 at a cost of roughly 2700 core-years. Deployed keys are 2048 bits or more, and the sieve's cost at that size exceeds any plausible computational budget.

7.3 From Factoring to Order Finding

The reduction proceeds in two steps. First, a non-trivial square root of 1 gives a factor. Second, a random choice of base produces such a square root with constant probability.

Theorem 7.6 (a non-trivial square root of unity reveals a factor). *Let N be composite (that is, not prime: $N = ab$ for some integers $a, b > 1$) and let y satisfy*

$$y^2 \equiv 1 \bmod N, \quad 1 < y < N - 1. \quad (52)$$

Then at least one of $\gcd(y-1, N)$ and $\gcd(y+1, N)$ is a non-trivial factor of N , and both greatest common divisors can be computed in $\mathcal{O}(L^2)$ bit operations.

Proof. From (52), N divides $y^2 - 1 = (y-1)(y+1)$. Suppose, for contradiction, that $\gcd(y-1, N) = 1$. Then N shares no factor with $y-1$, so N must divide $y+1$. But $1 < y < N-1$ gives $2 < y+1 < N$, and no integer strictly between 0 and N is divisible by N , a contradiction. Hence $\gcd(y-1, N) > 1$.

It remains to check that this common divisor is *proper*, i.e. not N itself. Since $0 < y-1 < N$, we have $\gcd(y-1, N) \leq y-1 < N$. So $\gcd(y-1, N)$ is a divisor of N strictly between 1 and N : a non-trivial factor. The same argument with the roles of $y-1$ and $y+1$ exchanged handles the symmetric case, and the cost is that of Euclid's algorithm. $\square$

The conditions $y \neq 1$ and $y \neq N - 1$ are not decoration. Both are square roots of 1 modulo N for every N , since $N - 1 \equiv -1$ and $(-1)^2 = 1$, and both are useless: they give $\gcd(0, N) = N$ and $\gcd(N, N) = N$ respectively. The content of the theorem is that any *other* square root of 1 is informative, and a composite N always has some, whereas a prime modulus has only ± 1 . Factoring is exactly the search for a non-trivial square root of unity.

So we need to manufacture such a y . The link to order finding is now one line. Suppose we pick x coprime to N and compute $r = \text{ord}_N(x)$. If r happens to be *even*, then

$$y = x^{r/2} \bmod N \quad (53)$$

is an integer satisfying $y^2 = x^r \equiv 1$, so it is a square root of 1. It cannot equal 1, because r is by definition the *least* exponent giving 1 and $r/2 < r$. It might equal $N - 1$, and that is the only remaining way to fail. So we need two lucky events, and the next theorem says that both occur with constant probability.

Theorem 7.7 (a random base usually works). *Let N be an odd composite integer with $m \geq 2$ distinct prime factors, $N = p_1^{\alpha_1} \cdots p_m^{\alpha_m}$. Let x be chosen uniformly at random from $\mathbb{Z}_N^*$ and let $r = \text{ord}_N(x)$. Then*

$$\Pr[r \text{ is even and } x^{r/2} \not\equiv -1 \bmod N] \geq 1 - \frac{1}{2^{m-1}}. \quad (54)$$

For $m = 2$, the RSA case $N = pq$, the bound is $1/2$, and it improves as N acquires more distinct prime factors. The full proof is a counting argument in the Chinese-remainder decomposition of $\mathbb{Z}_N^*$ and we do not reproduce it; see Nielsen and Chuang, Appendix 4.3. The following aside sketches why the number 2^{m-1} appears.

Where the bound comes from

By the Chinese remainder theorem, $\mathbb{Z}_N^* \cong \mathbb{Z}_{p_1^{\alpha_1}}^* \times \cdots \times \mathbb{Z}_{p_m^{\alpha_m}}^*$, and each factor is cyclic because each p_i is odd. So choosing x at random is the same as choosing m independent random elements x_i , one in each cyclic factor. Write $r_i = \text{ord}(x_i)$ and 2^{d_i} for the largest power of two dividing r_i ; then $r = \text{lcm}(r_1, \dots, r_m)$.

The algorithm fails in exactly two circumstances. It fails when r is odd, which requires $d_i = 0$ for *every* i . It fails when $x^{r/2} \equiv -1$, which requires $x_i^{r/2} \equiv -1$ in every factor simultaneously, and that forces all the d_i to be *equal* and nonzero. In a cyclic group of even order, a uniformly random element has d_i taking any particular value with probability at most $1/2$; so the probability that m independent draws all agree on a common value of d_i is at most $2^{-(m-1)}$. Both failure modes are of that form, which is the origin of (54).

7.4 The Algorithm

We can now write down the whole thing. Only step 5 uses a quantum computer; every other step runs classically and cheaply.

Algorithm 7.8 (Shor's factoring algorithm). *Input:* a composite integer N of L bits. *Output:* a non-trivial factor of N .

1. *Is N prime?* Test with the AKS or Miller–Rabin primality test, both classically efficient. If prime, report that there is nothing to factor and stop.
2. *Is N even?* If so, return the factor 2 and stop.
3. *Is $N = a^b$ for integers $a \geq 2$, $b \geq 2$?* There are only $\mathcal{O}(L)$ candidate exponents b , and for each one an integer b th root is found by binary search, so the test is classically efficient. If so, return a and stop.

4. *Choose a base.* Pick x uniformly at random in $\{2, \dots, N-1\}$ and compute $\gcd(x, N)$. If it exceeds 1 we have stumbled on a factor by luck: return it and stop. Otherwise $x \in \mathbb{Z}_N^*$ and we continue.
5. *Find the order.* Compute $r = \text{ord}_N(x)$, **using the quantum subroutine of Blocks 8 and 9**. If r is odd, or if $x^{r/2} \equiv -1 \pmod{N}$, declare failure and return to step 4 with a fresh x . Otherwise set $y = x^{r/2} \pmod{N}$.
6. *Extract the factor.* Compute $\gcd(y-1, N)$ and $\gcd(y+1, N)$ by Euclid's algorithm and return whichever is non-trivial.

Steps 1–3 exist to guarantee that Theorem 7.7 applies when we reach step 5: they dispose of the prime case, the even case, and the prime-power case, leaving an odd composite with $m \geq 2$ distinct prime factors. Step 5 then succeeds with probability at least $1/2$ per attempt by Theorem 7.7, so the expected number of attempts is at most two, and step 6 succeeds with certainty by Theorem 7.6.

To find *all* the prime factors of N , run the algorithm, divide, and recurse on both parts, using the primality test of step 1 to know when to stop. Since N has at most L prime factors counted with multiplicity, this multiplies the cost by at most L .

Example 7.9 (factoring 15 by hand, twice). Steps 1–3: 15 is not prime, not even, and not a perfect power. Continue.

First run, with $x = 7$. $\gcd(7, 15) = 1$, so we proceed. By Example 7.4, $r = \text{ord}_{15}(7) = 4$. It is even, so $y = 7^2 = 49 \equiv 4 \pmod{15}$, and $y = 4 \neq 14 = N-1$, so we do not fail. Then

$$\gcd(y-1, N) = \gcd(3, 15) = 3, \quad \gcd(y+1, N) = \gcd(5, 15) = 5,$$

and indeed $15 = 3 \times 5$. Both greatest common divisors happened to be informative here; in general only one need be.

Second run, with $x = 14$. $\gcd(14, 15) = 1$, so we proceed. But $14 \equiv -1$, so $14^2 \equiv 1$ and $r = 2$. It is even, giving $y = 14^1 = 14 = N-1$, precisely the excluded case. The run fails, we return to step 4, and we try again. This is not a flaw in the algorithm; it is the event whose probability Theorem 7.7 bounds. Of the eight units modulo 15, exactly one, namely 14, fails in this way.

7.5 What is Classically Easy, and Therefore Not the Bottleneck

It is worth collecting the classical ingredients, because a common misunderstanding is that factoring is hard “because arithmetic modulo N is hard”. It is not. Every one of the following is polynomial in L :

- addition modulo N : $\mathcal{O}(L)$. Multiplication and division modulo N : $\mathcal{O}(L^2)$ by schoolbook methods, and $\mathcal{O}(L \log L)$ by the best known;
- modular exponentiation $x^a \pmod{N}$ with $a < N$, by repeated squaring: at most $2L$ modular multiplications, hence $\mathcal{O}(L^3)$;
- the greatest common divisor, and hence modular inverses, by Euclid: $\mathcal{O}(L^2)$;
- primality testing: polynomial. The AKS test is deterministic, its best known variant running in $\mathcal{O}(L^6)$ up to logarithmic factors; in practice one uses the much faster randomised Miller–Rabin test;
- deciding whether $N = a^b$: polynomial, $\mathcal{O}(L^3)$ up to logarithmic factors.

These are the schoolbook figures, which is why $\mathcal{O}(L^3)$ will reappear as the cost of Shor's algorithm; fast multiplication improves several of them, and Section 9.5 records what that does to

the total. Exactly one operation on this list is missing, and it is the order. That single gap is what separates a laptop from a quantum computer, and closing it is the content of the next block.

Looking ahead. Block 8 shows that r is hiding in the eigenvalues of a unitary that we know how to build (multiplication by x modulo N), so that the whole of order finding reduces to the phase estimation of Lecture 8. The one genuine obstacle is that the eigenstates concerned cannot be prepared without already knowing r , and the way around it is the prettiest step in this course.

8 Block 8: Order Finding as Phase Estimation

Learning goals:

- Define the modular multiplication operator and prove it is unitary.
- Derive its eigenstates and eigenvalues, and see where r appears.
- Explain why those eigenstates cannot be prepared, and how the roots-of-unity lemma of Lecture 7 rescues the situation.
- Describe how controlled modular exponentiation is implemented and state its cost.
- Assemble the complete order-finding circuit and count its qubits and gates.

A short raison d'être

This block is short because Lecture 8 did the work. All that is needed is to find a unitary whose eigenvalues encode r , and to check the two hypotheses phase estimation requires: that we can apply controlled powers of it, and that we can prepare an eigenstate.

The first hypothesis is satisfied for a reason that is itself worth noticing. The unitary is multiplication modulo N , a classical arithmetic operation, and Lecture 6 established that any efficient classical computation can be run reversibly on a quantum computer without leaving garbage behind. So the quantum circuit's hardest component is a piece of ordinary arithmetic.

The second hypothesis fails outright, and the failure is instructive: the eigenstates depend on r , so preparing one presupposes the answer. The escape is a piece of algebra (Section 8.4) which says that although we cannot prepare any single eigenstate, their uniform superposition is the state $|1\rangle$, which costs one X gate.

8.1 The Shape of the Quantum Part

Before any of the algebra, here is the whole quantum subroutine in one picture. It uses two registers: a *counting* register of t qubits, whose measured value will be the answer, and a *work* register of L qubits, on which the arithmetic happens. Figure 8 follows both of them through the four stages of the circuit.

Everything in this block is the justification of that middle equality. It needs three ingredients, and the next three sections supply them in turn: a unitary to call U_x (Section 8.2), its eigenstates $|u_s\rangle$ and eigenvalues $e^{2\pi i s/r}$ (Section 8.3), and the observation that $|1\rangle$ is the equal superposition of those eigenstates (Section 8.4), which is why the second register may be started in a state

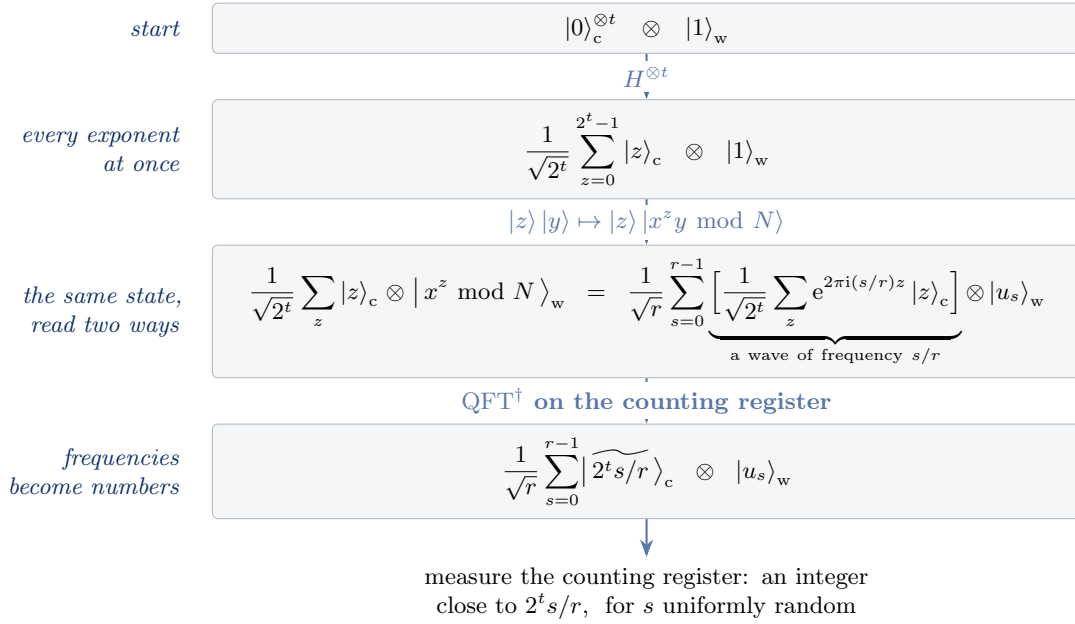


Figure 8: Order finding, before any of the details. The only step that is not routine is the middle equality, and it carries the whole algorithm: the modular exponentiation does not merely compute x^z , it splits the state into r branches, and in the s th branch the counting register has been stamped with a wave of frequency s/r . An inverse Fourier transform reads a frequency off a register, so the last two stages simply collect the answer.

that costs a single gate. Section 8.5 then builds the arrow labelled $x^z \bmod N$ out of ordinary arithmetic.

A second way to see the same thing, which some find easier and which needs no eigenvalues at all. Suppose that at the middle stage we *measured* the work register, obtaining some value $x^{z_0} \bmod N$. The counting register would collapse onto those z giving that value, namely $z_0, z_0 + r, z_0 + 2r, \dots$: a comb of spacing r with an unknown offset z_0 . That is exactly the state whose Fourier transform Lecture 7, Section 2.3 computed, and the transform turns it into a comb of spacing $2^t/r$ with the offset hidden in a phase. The eigenvalue language and the period language describe the same circuit; we use the first because it connects directly to Lecture 8, and because it does not require the imaginary measurement.

8.2 The Multiplication Operator

Fix N and x with $\gcd(x, N) = 1$, and let the work register hold L qubits, so that it spans $\{|0\rangle, |1\rangle, \dots, |2^L - 1\rangle\}$ with $2^L \geq N$.

Definition 8.1 (modular multiplication). Let U_x act on the computational basis of the work register by

$$U_x |y\rangle = \begin{cases} |xy \bmod N\rangle, & 0 \leq y < N, \\ |y\rangle, & N \leq y < 2^L. \end{cases} \quad (55)$$

The second line is a convention, not a mathematical necessity. The register is large enough to hold values up to $2^L - 1$, which may exceed $N - 1$; those extra basis states must be sent somewhere, and sending each to itself is the cheapest choice. They play no role in the algorithm, since the states we shall actually use are supported on $\{0, \dots, N - 1\}$.

Proposition 8.2. U_x is unitary.

Proof. It suffices to show that U_x permutes the computational basis, since a permutation matrix has orthonormal columns. On the range $N \leq y < 2^L$ it is the identity. On $\{0, \dots, N-1\}$ it is the map $y \mapsto xy \bmod N$, which is injective: if $xy \equiv xy'$ then $x(y - y') \equiv 0$, and multiplying by the inverse x^{-1} , which exists because $\gcd(x, N) = 1$, gives $y \equiv y'$, hence $y = y'$ as both lie in $\{0, \dots, N-1\}$. An injective map of a finite set to itself is a bijection. So U_x permutes the basis and is unitary. $\square$

The hypothesis $\gcd(x, N) = 1$ is doing real work here: without an inverse, multiplication by x is not injective, and the operator is not unitary. This is the same coprimality that step 4 of Algorithm 7.8 checks, and it is no coincidence.

8.3 Eigenstates and Eigenvalues

Theorem 8.3 (spectrum of U_x on the orbit of 1). *Let $r = \text{ord}_N(x)$ and, for $s = 0, 1, \dots, r-1$, define*

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |x^k \bmod N\rangle. \quad (56)$$

These r states are orthonormal and satisfy

$$U_x |u_s\rangle = e^{2\pi i s / r} |u_s\rangle. \quad (57)$$

Proof. The r integers $x^0, x^1, \dots, x^{r-1}$ modulo N are distinct: if $x^k \equiv x^{k'}$ with $0 \leq k < k' < r$ then $x^{k'-k} \equiv 1$ with $0 < k' - k < r$, contradicting minimality of r . So the kets in (56) are r distinct computational basis states. Orthonormality of the $|u_s\rangle$ then follows from Lemma 2.3 applied with modulus r : $\langle u_{s'} | u_s \rangle = \frac{1}{r} \sum_k e^{2\pi i (s' - s)k / r} = \delta_{ss'}$.

For the eigenvalue equation, apply U_x to (56). Since $x \cdot x^k \equiv x^{k+1}$,

$$U_x |u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |x^{k+1}\rangle = \frac{1}{\sqrt{r}} \sum_{k'=1}^r e^{-2\pi i s (k'-1) / r} |x^{k'}\rangle = e^{2\pi i s / r} \frac{1}{\sqrt{r}} \sum_{k'=1}^r e^{-2\pi i s k' / r} |x^{k'}\rangle.$$

The remaining sum runs from $k' = 1$ to r rather than 0 to $r-1$, but the $k' = r$ term equals the $k' = 0$ term: $x^r \equiv x^0 = 1$ and $e^{-2\pi i s r / r} = e^{-2\pi i s} = 1 = e^{-2\pi i s \cdot 0 / r}$. So the sum is again $\sqrt{r} |u_s\rangle$, which gives (57). $\square$

Look at what (57) says. The eigenvalue is $e^{2\pi i \varphi}$ with

$$\varphi = \frac{s}{r}. \quad (58)$$

The quantity we want is sitting in the denominator of a phase. Phase estimation reads phases. That is the entire idea of the algorithm, and everything remaining is the removal of obstacles.

Remark 8.4. U_x has other eigenvectors. The basis states $|y\rangle$ with $N \leq y < 2^L$ are fixed, hence eigenvectors with eigenvalue 1, as are suitable combinations on the other orbits of multiplication by x inside $\mathbb{Z}_N^*$. None of them will appear, because the algorithm will only ever feed U_x states supported on the orbit of 1.

8.4 The Preparation Trick

Phase estimation requires an eigenstate as input. Inspect (56): to build $|u_s\rangle$ we would need to know how many terms the sum has, and how fast the phases rotate, both of which are r . Preparing an eigenstate presupposes the answer we are trying to compute. As a subroutine this is worthless.

Here is the escape.

Theorem 8.5 (the eigenstates sum to a state we can prepare). *With $|u_s\rangle$ as in (56),*

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = |1\rangle. \quad (59)$$

Proof. Substitute (56) and exchange the two finite sums:

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = \frac{1}{r} \sum_{s=0}^{r-1} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |x^k\rangle = \frac{1}{r} \sum_{k=0}^{r-1} \left(\sum_{s=0}^{r-1} e^{-2\pi i s k / r} \right) |x^k\rangle.$$

The inner sum is exactly Lemma 2.3 with modulus r and $a = -k$: it equals r when $k \equiv 0$ and 0 otherwise. Since k runs over $0, \dots, r-1$, only $k = 0$ survives, leaving $\frac{1}{r} \cdot r \cdot |x^0\rangle = |1\rangle$. $\square$

This deserves a moment. Each individual $|u_s\rangle$ is a highly entangled state, spread over r basis labels that are scattered unpredictably through $\{0, \dots, N-1\}$, and we cannot construct any one of them. Their uniform superposition, however, is $|1\rangle$: a computational basis state, prepared from $|0 \cdots 0\rangle$ by a single X gate on the last qubit. Every phase in every $|u_s\rangle$ cancels against its partners except at $k = 0$, and the whole entangled structure collapses to one bit set to one.

And now Lecture 8, Section 5.4 pays for itself. Phase estimation run on a superposition $\sum_u c_u |u\rangle$ of eigenstates returns the phase φ_u with probability $|c_u|^2$. Feeding it $|1\rangle = r^{-1/2} \sum_s |u_s\rangle$ therefore returns an estimate of

$$\varphi = \frac{s}{r} \quad \text{for uniformly random } s \in \{0, 1, \dots, r-1\},$$

with the work register left in the corresponding $|u_s\rangle$. We never learn which s we got, and we never needed to. What we did not know, namely which eigenstate, has been converted into harmless randomness, exactly as the unknown offset x_0 was in Simon's algorithm (Lecture 7, Section 1.6) and in the Fourier transform of a periodic vector (Lecture 7, Section 2.3).

8.5 Controlled Modular Exponentiation

The second hypothesis of phase estimation is that we can apply the controlled operations $C \cdot U_x^{2^j}$ for $j = 0, 1, \dots, t-1$. Note first what these operations *are*:

$$U_x^{2^j} |y\rangle = |x^{2^j} y \bmod N\rangle, \quad (60)$$

that is, multiplication by the constant $x^{2^j} \bmod N$. That constant is computed once, in advance, on a classical computer by repeated squaring:

$$x, \quad x^2, \quad x^4 = (x^2)^2, \quad x^8 = (x^4)^2, \quad \dots$$

each step being one modular squaring at $\mathcal{O}(L^2)$ bit operations, for $\mathcal{O}(L^3)$ in total over all $t = \mathcal{O}(L)$ values of j . So the quantum circuit never has to *compute* a power; it has to multiply by a number that is already known.

That is a substantial simplification, and there is a second one. Consider what the whole controlled sequence does to the state of the two registers. Writing the counting register in the computational basis, the sequence maps

$$|z\rangle_c |y\rangle_w \mapsto |z\rangle_c |x^z y \bmod N\rangle_w, \quad (61)$$

since the controlled operations apply $U_x^{2^j}$ exactly when bit j of z is set, and $\prod_j (x^{2^j})^{z_j} = x^z$. The whole block is therefore *modular exponentiation*, and it is this, not the Fourier transform, that dominates the cost of Shor's algorithm. But we only ever apply it with $y = 1$, by Theorem 8.5, so the circuit does not need to implement (60) for arbitrary y : any unitary agreeing with it on $y = 1$ will do, which leaves the implementer useful freedom.

How is a modular multiplication built as a circuit? Exactly as Lecture 6 prescribed. Multiplication of an L -bit register by a known constant modulo a known modulus is an ordinary classical algorithm of $\mathcal{O}(L^2)$ elementary operations; compile it into Toffoli and CNOT gates, run it on ancillas, copy out the answer, and uncompute to return the ancillas to $|0\rangle$ so that no garbage remains entangled with the computation. Lecture 6 showed that this costs a constant factor over the classical circuit and produces a genuine unitary. Summing $\mathcal{O}(L^2)$ over the $t = \mathcal{O}(L)$ controlled operations gives

$$\mathcal{O}(L^3) \text{ gates for the modular exponentiation block.} \quad (62)$$

Remark 8.6. Uncomputation is not an optional tidiness. If the ancillas were left holding intermediate results, they would remain entangled with the counting register, and the interference that the inverse Fourier transform is about to perform would be destroyed: the register would decohere into a classical mixture and phase estimation would return noise. This is the concrete reason Lecture 6 insisted on the point.

8.6 The Complete Circuit

Putting the three ingredients together (prepare $|1\rangle$, apply controlled modular exponentiation, invert the Fourier transform) gives the order-finding circuit.

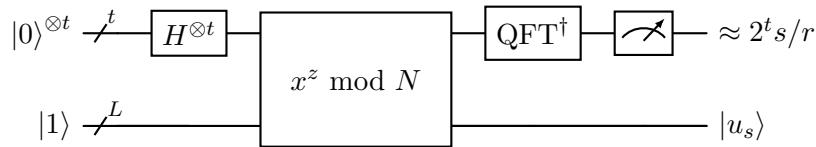


Figure 9: Order finding. The counting register of t qubits is put into uniform superposition; the central block performs $|z\rangle |y\rangle \mapsto |z\rangle |x^z y \bmod N\rangle$ as in (61); the inverse quantum Fourier transform of Lecture 7, Section 3.5 converts the accumulated phases into a number, which the measurement reads out. The result is a t -bit approximation to $2^t s / r$ for a uniformly random $s \in \{0, \dots, r - 1\}$.

Resource summary for order finding.

Component	Qubits	Gates
counting register	$t = 2L + \mathcal{O}(1)$	–
work register	L	–
ancillas for arithmetic	$\mathcal{O}(L)$	–
Hadamard layer	–	$\mathcal{O}(L)$
modular exponentiation	–	$\mathcal{O}(L^3)$
inverse Fourier transform	–	$\mathcal{O}(L^2)$, or $\mathcal{O}(L \log L)$ approximate
Total	$\mathcal{O}(L)$	$\mathcal{O}(L^3)$

The arithmetic dominates by a factor of L . It is worth remembering which part of Shor’s algorithm is expensive, because it is not the part with the famous name.

The value of t has been left as $2L + \mathcal{O}(1)$ without justification. Fixing it requires knowing how accurate an estimate of s/r must be before r can be recovered from it, which is a question about continued fractions and therefore the first business of the next block.

Looking ahead. The measurement returns an integer close to $2^t s/r$, from which we can compute a real number close to s/r . What we want is r . Recovering the denominator of an unknown fraction from a decimal approximation to it sounds impossible, since every real number is close to fractions with every conceivable denominator. It is possible only because $r < N$ bounds the denominator in advance. The tool is two thousand years old and is called the continued fraction expansion.

9 Block 9: From a Phase to an Integer, and What It All Means

Learning goals:

- Compute a continued fraction expansion and its convergents.
- State the approximation theorem that justifies the choice $t \approx 2L$.
- Enumerate the ways the algorithm can fail and say how each is detected and repaired.
- Run the complete algorithm on a worked example with a non-dyadic phase.
- Place Deutsch–Jozsa, Simon and Shor inside the hidden subgroup problem, and say where that framework stops working.
- State honestly what it would cost to run Shor’s algorithm on a cryptographically relevant number.

A short raison d’être

There is a step in Shor’s algorithm that is easy to skip over and that is, in its own way, as surprising as anything quantum in these notes. The machine hands us a real number: an approximation, accurate to some number of bits, of the fraction s/r . Both s and r are unknown. We want r .

Stated baldly this is impossible. Every real number is within 10^{-6} of fractions with denominator 10^7 , denominator 10^8 , and so on without end; an approximation alone cannot single out a denominator. What makes it possible is one extra piece of information: we

know in advance that $r < N$. Among fractions with denominator below N , a sufficiently good approximation determines the fraction *uniquely*, and there is a classical algorithm that finds it. That is the content of this block's first two sections, and it completes the algorithm.

9.1 Continued Fractions

Definition 9.1 (continued fraction expansion). Every positive real number φ has an expansion

$$\varphi = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \dots}}} =: [a_0; a_1, a_2, a_3, \dots] \quad (63)$$

with $a_0 = \lfloor \varphi \rfloor$ and the remaining a_i positive integers, obtained by the algorithm: set $\varphi_0 = \varphi$; at each step put $a_i = \lfloor \varphi_i \rfloor$ and $\varphi_{i+1} = 1/(\varphi_i - a_i)$, stopping when $\varphi_i = a_i$. The expansion terminates if and only if φ is rational. The i th *convergent* is the fraction $h_i/k_i = [a_0; a_1, \dots, a_i]$, computed by the recurrences

$$h_i = a_i h_{i-1} + h_{i-2}, \quad k_i = a_i k_{i-1} + k_{i-2}, \quad (64)$$

started from $h_{-1} = 1$, $h_{-2} = 0$, $k_{-1} = 0$, $k_{-2} = 1$.

For a rational input p/q the algorithm is Euclid's algorithm on (p, q) in disguise, so it costs $\mathcal{O}(L^3)$ and runs in $\mathcal{O}(L)$ steps. The convergents are, in a precise sense, the best rational approximations to φ available at their denominator: no fraction with a smaller denominator comes closer.

Example 9.2 (the expansion we shall need). Take $\varphi = 341/2048 = 0.166503906\dots$, which is what an 11-bit phase estimate of $1/6$ looks like. Then:

step i	a_i	convergent h_i/k_i	value
0	0	0/1	0.000000
1	6	1/6	0.166667
2	170	170/1021	0.166503
3	2	341/2048	0.166504

The expansion terminates at the input itself, as it must for a rational number. The useful line is the second: $1/6$ approximates φ to within 1.6×10^{-4} using a denominator of 6, while the next convergent buys an improvement of less than 10^{-6} at the cost of a denominator of 1021. That abrupt jump in the denominator is the signature of the fraction we are looking for, and the stopping rule below formalises it.

9.2 How Accurate Is Accurate Enough

Theorem 9.3 (continued fractions recover a fraction from a good enough approximation). *Let s/r be a rational number and φ a real number with*

$$\left| \frac{s}{r} - \varphi \right| \leq \frac{1}{2r^2}. \quad (65)$$

Then s/r is a convergent of the continued fraction expansion of φ , and is therefore computable from φ in $\mathcal{O}(L^3)$ operations.

We use the theorem without proof; it is a classical result of Diophantine approximation (Nielsen and Chuang, Theorem A4.16, following Hardy and Wright). Its content is a uniqueness statement: condition (65) is tight enough that *no other* fraction with denominator at most r can also satisfy it, so the convergent is unambiguous.

Now we can size the counting register. We want (65) to hold for the unknown r , and all we know about r is $r < N$. So it suffices to have

$$\left| \frac{s}{r} - \varphi \right| \leq \frac{1}{2N^2}, \quad (66)$$

which, since $2^L \geq N$, is implied by an accuracy of $2^{-(2L+1)}$, that is, by knowing φ to $2L + 1$ bits. By Lecture 8, Theorem 5.2, obtaining n correct bits with probability at least $1 - \epsilon$ needs a counting register of $t = n + \lceil \log_2(2 + 1/2\epsilon) \rceil$ qubits. Putting $n = 2L + 1$ and holding the failure probability ϵ fixed,

$$t = (2L + 1) + \left\lceil \log_2 \left(2 + \frac{1}{2\epsilon} \right) \right\rceil = 2L + \mathcal{O}(1), \quad (67)$$

as promised in Block 8. The register is of linear size: the number of qubits is proportional to the number of bits of the number being factored, which is the least one could hope for.

9.3 Failure Modes, and How to Detect Them

The algorithm as it stands is probabilistic in three separate places, and it is worth listing them together because each is handled differently.

1. Phase estimation may return a poor estimate. By Lecture 8, Theorem 5.1, the measured value is the best t -bit approximation with probability at least $4/\pi^2 \approx 0.405$, and (67) converts the remaining risk into the tunable ϵ . This failure is *not* directly detectable, but it is caught by the check in item 3.

2. s and r may share a factor. The measurement gives an approximation to s/r with s uniformly random in $\{0, \dots, r - 1\}$. Continued fractions return this fraction *in lowest terms*, that is s'/r' with $s' = s/g$, $r' = r/g$ and $g = \gcd(s, r)$. When $g > 1$ we obtain a proper divisor r' of r , not r itself. (The case $s = 0$, which occurs with probability $1/r$, gives no information at all.)

This is repaired by repetition. The estimate is correct whenever $\gcd(s, r) = 1$, and the number of integers below r coprime to r is $\phi(r)$, which is at least of order $r / \log \log r$, so a random s is coprime to r with probability at least of order $1 / \log \log r$. Hence $\mathcal{O}(\log \log r) = \mathcal{O}(\log L)$ repetitions suffice. In practice one does better: running the algorithm twice and taking the least common multiple of the two denominators succeeds with constant probability, which brings the repetition count down to $\mathcal{O}(1)$.

3. The check. Whatever candidate r' emerges, it is verified classically in $\mathcal{O}(L^3)$ operations by computing $x^{r'}$ mod N and testing whether it equals 1. If it does, we have an exponent that works and we proceed; if not, we repeat. This single test catches both failure 1 and failure 2, and it means that the algorithm never returns a wrong answer, only, occasionally, no answer. An algorithm that can verify its own output cheaply is in a very comfortable position, and this one can.

4. And the base may be unlucky , as Theorem 7.7 and Example 7.9 already described: r odd, or $x^{r/2} \equiv -1$. Detected trivially, repaired by drawing a new x , and occurring with probability at most $1/2$.

9.4 A Complete Worked Example

Take $N = 21$, so $L = \lceil \log_2 21 \rceil = 5$ and $t = 2L + 1 = 11$, giving a Fourier transform of dimension $2^{11} = 2048$. Steps 1–3 of Algorithm 7.8 pass: 21 is odd, composite, and not a perfect power. Suppose step 4 draws $x = 2$, with $\gcd(2, 21) = 1$.

The order is $r = \text{ord}_{21}(2) = 6$, since $2^6 = 64 = 3 \cdot 21 + 1$. The algorithm does not know this; the quantum subroutine is about to discover it. The eigenvalue phases are $s/6$ for $s = 0, \dots, 5$, and *none but $s = 0$ and $s = 3$ is exactly representable in 11 bits*. This is the generic, leaky situation of Lecture 7, Example 2.5, not an artificially clean one.

Suppose the measurement returns the integer 341, so that

$$\varphi = \frac{341}{2048} = 0.166503906\dots$$

This is the most likely outcome when $s = 1$: it is the nearest 11-bit value to $1/6 = 0.1\bar{6}$, and phase estimation returns it with probability about 0.68. The continued fraction expansion is the one computed in Example 9.2. We take convergents in turn and stop at the last one whose denominator is less than $N = 21$: the convergents are $0/1$, $1/6$, $170/1021$, $341/2048$, and the denominators 1021 and 2048 are far too large. So the algorithm outputs

$$r' = 6.$$

Check it classically: $2^6 = 64 \equiv 1 \pmod{21}$. It passes.

Now finish on the laptop. $r' = 6$ is even, so $y = 2^3 = 8$, and $8 \neq 20 = N - 1$, so we have not failed. Then

$$\gcd(y - 1, N) = \gcd(7, 21) = 7, \quad \gcd(y + 1, N) = \gcd(9, 21) = 3,$$

and $21 = 3 \times 7$.

Remark 9.4 (why not $N = 15$?). The example usually given is $N = 15$, and it is worth saying why we have not used it here. For $N = 15$ every base has order $r \in \{1, 2, 4\}$, a power of two, so every phase s/r is exactly representable in binary, phase estimation returns it with probability 1, and the continued fraction step is trivially exact. The example therefore exercises none of the machinery of this block. It is a genuinely degenerate case, and it is also the reason to be sceptical of demonstrations: see Section 9.7.

Remark 9.5 (an instructive failure). Suppose instead the measurement had returned 1024, giving $\varphi = 1024/2048 = 1/2$ exactly. This is the outcome for $s = 3$, and $\gcd(3, 6) = 3$, so the fraction in lowest terms is $1/2$ and the algorithm proposes $r' = 2$. The check fails: $2^2 = 4 \not\equiv 1 \pmod{21}$. We discard the run and repeat. This is failure mode 2 of Section 9.3, caught by the test of mode 3, at the cost of one more run.

9.5 The Cost, and the Comparison

Collecting the counts: one run of order finding costs $\mathcal{O}(L^3)$ gates (equation (62)), $\mathcal{O}(1)$ runs suffice on average, the classical post-processing is $\mathcal{O}(L^2)$, and the outer loop over choices of x multiplies by $\mathcal{O}(1)$. Writing $T_{\text{Shor}}(L)$ for the total cost of factoring an L -bit number,

$$T_{\text{Shor}}(L) = \mathcal{O}(L^3), \tag{68}$$

reducible to $\mathcal{O}(L^2 \log L \log \log L)$ if fast multiplication is used for the modular arithmetic. Against the sub-exponential (51) of the number field sieve, this is a super-polynomial improvement. It is not, as is sometimes said, an exponential improvement over the best classical algorithm, since the sieve is already sub-exponential, though it is an exponential improvement over trial division.

9.6 The General Picture: the Hidden Subgroup Problem

Step back from the arithmetic. We have now seen four algorithms with, it turns out, one structure.

Definition 9.6 (hidden subgroup problem). Let G be a group and $f : G \rightarrow S$ a function into some set, with the promise that there is a subgroup $H \leq G$ on which the following holds: f is constant on each *coset* of H , that is, on each set $g + H = \{g + h : h \in H\}$ obtained by shifting H by some $g \in G$, and takes distinct values on distinct cosets. Find H . (In Simon’s problem G is the set of n -bit strings under $\oplus$, the subgroup is $H = \{0, c\}$, and its cosets are exactly the pairs $\{x, x \oplus c\}$ on which Lecture 7 found the state (8) to be supported.)

Algorithm	Group G	Hidden subgroup H
Deutsch–Jozsa, Bernstein–Vazirani	$(\mathbb{Z}_2)^n$	determined by the hidden string
Simon	$(\mathbb{Z}_2)^n$	$\{0, c\}$
Order finding, and hence Shor	$\mathbb{Z}$	$r\mathbb{Z}$
Discrete logarithm	$\mathbb{Z}_{p-1} \times \mathbb{Z}_{p-1}$	generated by $(\ell, -1)$

And the solution is always the same four steps, which we wrote down in Lecture 7 at the end of Section 1.6: prepare a uniform superposition over G , query f once so that the first register is supported on one coset of H , apply the Fourier transform *of the group G* to convert the unknown shift g into an irrelevant global phase, and measure to obtain a random constraint on H . The only thing that changes from row to row is which Fourier transform is needed, and whether we know how to build it efficiently.

For every *abelian* G we do, and the hidden subgroup problem is solved. This is the correct statement of what quantum computers are known to be good at, and it is narrower than the advertising suggests.

For non-abelian G the Fourier transform still exists, built from the group’s irreducible representations, but the final measurement no longer yields usable constraints, and no efficient algorithm is known. Two cases are worth naming because of what hangs on them. Graph isomorphism is the hidden subgroup problem for the symmetric group; and the shortest-vector problem in a lattice reduces to the hidden subgroup problem for the dihedral group. The second is why post-quantum cryptography has largely bet on lattices: they sit in exactly the region where the technique that breaks RSA is known not to reach.

9.7 Reality Check

Three sobering observations, which are also the bridge to the remainder of the course.

The resource estimates. Equation (68) counts *logical* gates on a perfect machine. A real device is noisy, and Lecture 8, Section 6.4 already noted that phase estimation needs a coherent circuit of depth $\mathcal{O}(2^t)$ applications of U_x . For $L = 2048$ that is billions of gates in sequence, with no measurable error permitted in any of them. This is unreachable without error correction. Detailed estimates for RSA-2048 have come down steadily as the constructions improved: from roughly twenty million noisy physical qubits running for eight hours,⁵ to under one million in the most recent work.⁶ Devices today have on the order of a thousand physical qubits. The gap is one of the main subjects of Lectures 11–13.

⁵C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”, *Quantum* **5**, 433 (2021); arXiv:1905.09749.

⁶C. Gidney, “How to factor 2048 bit RSA integers with less than a million noisy qubits”, arXiv:2505.15917 (2025).

Small demonstrations prove very little. Numbers such as 15, 21 and 35 have been “factored” on quantum hardware since 2001. As Example 7.9 and the remark in Section 9.4 showed, these instances are degenerate: the orders involved are tiny and often powers of two, and the circuits reported are usually compiled with knowledge of the answer, which removes precisely the parts that make the algorithm hard. A well-known critique makes the point sharply and is worth reading.⁷ When you read a factoring record, ask what r was and whether the circuit could have been built without knowing it.

And yet it already matters. Encrypted traffic recorded today can be stored and decrypted later, so any secret whose value outlives the arrival of a large quantum computer is already exposed: the “harvest now, decrypt later” problem. This is why standardisation did not wait: NIST published its first post-quantum standards in August 2024, based on lattice and hash constructions rather than on factoring or discrete logarithms, and migration is under way. The correct summary is that Shor’s algorithm has already changed cryptography, and has not yet run on anything.

Where this leads

It is worth stating plainly what these three lectures did and did not establish.

They established that a quantum computer can do something a classical computer is not known to do: find the period of a function, and thereby factor integers, in polynomial time. They established that the mechanism is neither parallelism nor magic but a Fourier transform, which converts an unknown offset into an invisible phase and a hidden period into the location of a peak. And they established, in Lecture 7, Section 3.7 and again in Section 9.7, that this mechanism applies only to problems with cheap read-in and cheap read-out, which is a small class.

What they did not establish is that any of it can be run. Every circuit in these three lectures assumed perfect gates, and every one of them is far too deep for a device whose qubits decohere after a few hundred operations, as Lecture 2 described and as the numbers in Section 9.7 confirm. The algorithms are finished; the machine is not.

That gap is the subject of the rest of the course. Lecture 10 describes what noise actually does to a quantum computation and what can be salvaged by mitigating it without correcting it. Lectures 11 to 13 build the only known real answer: quantum error correction, which uses many imperfect physical qubits to simulate a few very good logical ones, and which turns every resource count in this lecture into a much larger one. The algorithms of Lectures 7 to 9 are the reason that programme is worth the effort.

⁷J. A. Smolin, G. Smith and A. Vargo, “Oversimplifying quantum factoring”, *Nature* **499**, 163 (2013). The authors factor a 20000-bit number with a two-qubit circuit, by choosing the number appropriately.